

GRAFOMEM Cloud — Internal Technical Whitepaper

Classification: INTERNAL — Not for publication Version: 2.6.6 · June 2026 Authors: GNS Foundation Engineering

1. Executive Summary

GRAFOMEM Cloud is a **governed agent memory platform** where every AI inference decision is logged, signed, and replayable, every memory operation is policy-gated, and every data deletion produces a cryptographic erasure certificate.

The platform is built on **7 governance layers** stacked on top of the open-source GMP (Grafomem Memory Protocol) specification. The open spec (MIT, frozen at v0.2.0) defines what a memory store must do; the Cloud platform enforces *how* it does it — with provenance, compliance, and auditability at every step.

[!TIP] **v2.6 milestone**: Sprint 21 delivers the **Semantic Manifold (Visual Governance)** architecture. The orchestrator UI has been fully rewritten into a WebGL/d3-canvas Level 1 / Level 2 deep zoom interface, visually mapping the high-dimensional execution space into an interactive, compliance-filtered hexagonal SOM grid.

v2.4/2.5 milestone: Sprints 17–20 deliver **multi-provider LLM support** — all three providers now have a live conformance run (OpenAI gpt-4o-mini at the 39-test baseline; Anthropic Claude Opus 4 at **49/51**; Google Gemini 2.5 Pro at **48/51** — no live run is a clean 51; see §32) — plus **CrewAI/AutoGen SDK adapters**, **Continuous Assurance** (drift detection, scheduled checks, baselines), and **RoutingPool** (read-replica routing). Platform now has **~163 API endpoints**, **112/112 unit tests green**, and **113 conformance gates** (mock). v2.5 is a consistency reconciliation pass — endpoint/table counts now cite `openapi.json` and the schema as the source of truth; §16, §18, §21, and §2 are brought current through Sprint 20.

v2.6.2 adds Sprint 22: **Integration Seam Regression Net** — 13 integration tests against real local Postgres (no mocks) covering RBAC, key rotation, loop detection, timeout, SSE status, and erasure wiring. Five infrastructure bugs fixed during bringup: `_PooledConnectionProxy.__setattr__` (broke pooled transactional

connections), gcrumbs transaction rollback (advisory lock deadlocks), `_run_round_robin` missing `FAILED_TIMEOUT` handler (deadline-exceeded workflows not terminated), `stream_workflow` route referencing undefined `req` (SSE streaming broken), and `termination_reason` wired to Workflow dataclass + DB + API + SSE.

v2.6.3 adds Sprint 23: **Scoped / Role-Based Keys** — flat single-key-per-tenant replaced with scoped, prefixed, least-privilege API keys. 14-scope vocabulary (`memory:read/write/admin`, `orchestrator:run/admin`, `governance:read/admin`, `decisions:read`, `erasure:execute`, `gcrumbs:read`, `llm:admin`, `webhooks:admin`, `keys:admin`, *). Key prefixes (`gfm_`, `gfm_ro_`, `gfm_sa_`). Scope enforcement wired into all 6 route files (30+ endpoints). Per-key `expires_at`, `allowed_stores`, `ip_allowlist`, `last_used_at`. Test suite at 26/26 (13 new scope tests, zero regressions).

v2.6.4 adds Sprint 23b: **Scope Perimeter Closure** — the scope audit expanded enforcement from 6 to **all 22 route files** and the vocabulary from 14 to **21 scopes** (added `admin:platform`, `compliance:read/admin`, `artifacts:read/admin`, `manifold:read`, `sso:admin`). Every authenticated endpoint now resolves to a defined scope. LLM provider routes (holding API keys) require `llm:admin`; key-management endpoints require `keys:admin` (privilege-escalation guard). `revoke_key` now invalidates the TTL cache immediately. `ip_allowlist` enforced at auth time (source IP checked → 403 on mismatch). Two-sided (deny+allow) tests for every scoped surface. Test suite at **58/58** (32 new tests, zero regressions).

v2.6.5 adds Sprint 24: **Self-Serve Commercialization & Stripe Billing** — The platform is now commercially ready with full Stripe integration. The backend Python orchestrator implements secure `create_checkout_session` and `create_portal_session` handlers, managed via robust Webhook listeners that handle lifecycle events (`checkout.session.completed`, `customer.subscription.deleted`, etc.). The React Next.js Portal (`portal.grafomem.com`) has been wired to support self-serve upgrades and billing management directly from the tenant Settings UI.

v2.6.6 adds Phase 7: **Scale, Security, and Sync** — Deployed cross-node DEK cache invalidation using Postgres `pg_notify` for sub-500ms multi-node sync. Swept all API boundaries for HTTP 500 state leakage, sanitizing internal server errors. Verified full Enterprise Event Webhooks deployment (HMAC-SHA256 signatures, asynchronous dispatch) guaranteeing immediate, secure downstream compliance propagation. Completed the formal vault penetration test and generated the enterprise security checklist. **v2.6.1** is a status-reconciliation pass: the Appendix live-status table and the §17.1 critical-path rows are brought current with shipped

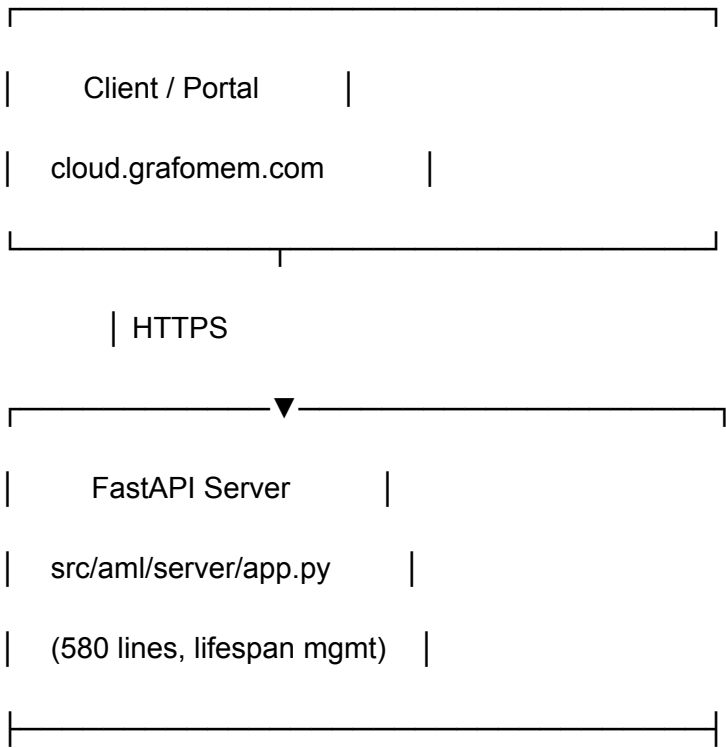
functionality (gcrumbs + signed erasure live since Sprint 15; the resilience mechanisms — failover, tool-deny, timeout, loop — validated two-sided in the sealed run); §17.2 conformance counts are aligned to §32 (51/51 mock; OpenAI 39-test baseline); and competitive-absolute language ("the only") and one "tamper-proof" slip are corrected to defensible claims ("tamper-evident").

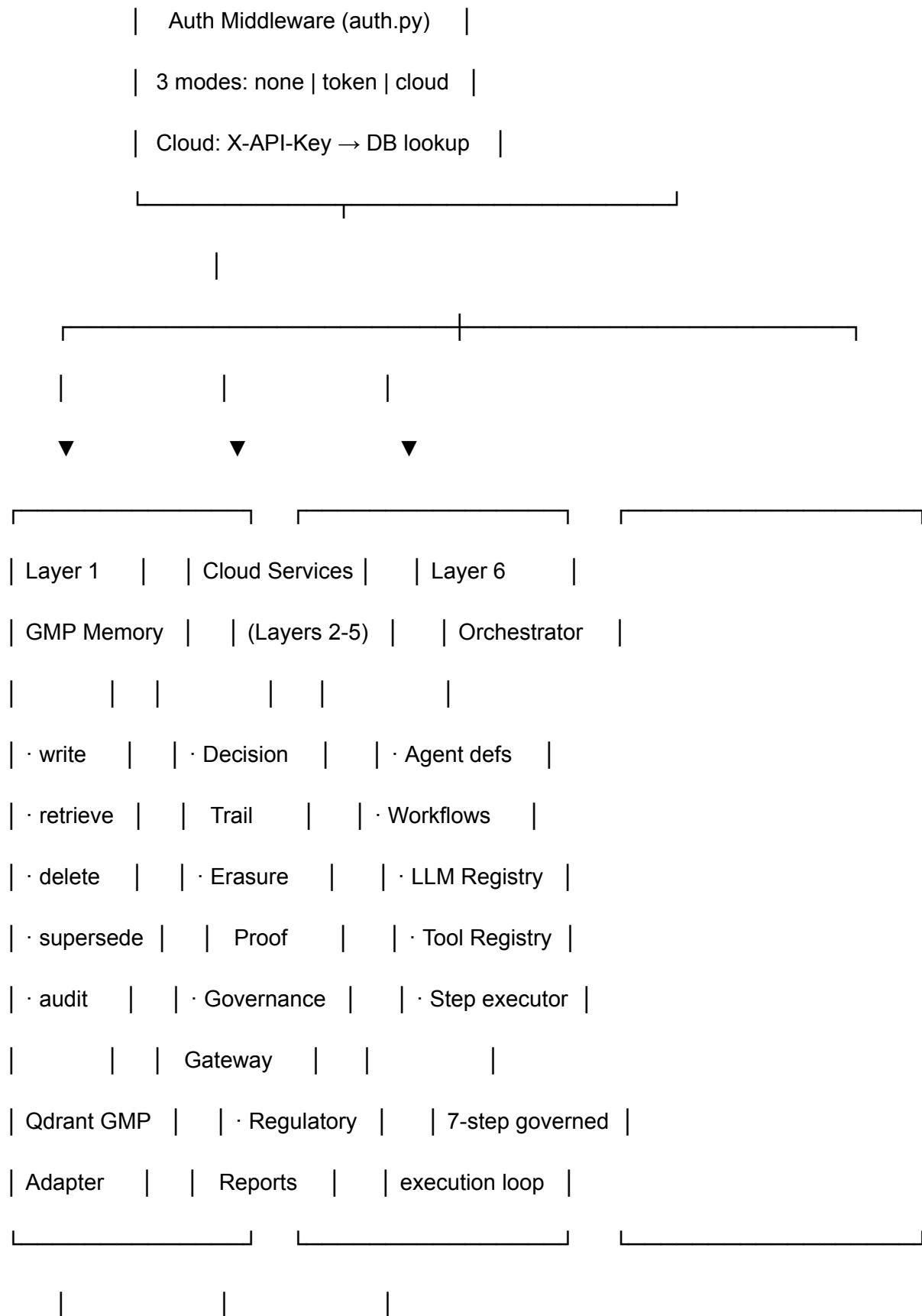
Key Numbers

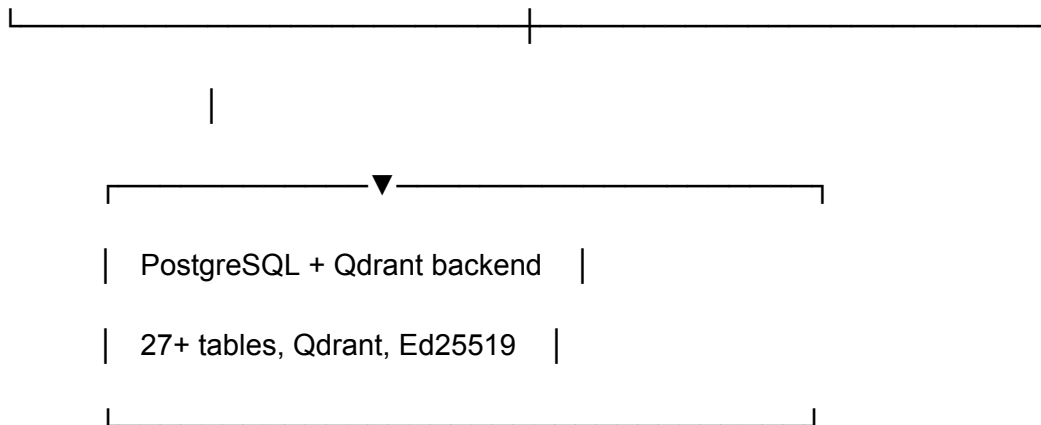
Metric	Value
Total Python source	~39,000 lines
Cloud modules (<code>src/aml/cloud/</code>)	~22,800 lines across 63 files
Python SDK (<code>sdk/</code>)	~3,000 lines across 26 files (incl. CrewAI + AutoGen adapters)
Portal UI	~5,200 lines (React + HTML + CSS + JS)
API endpoints	~163 — authoritative count is the exported <code>docs/openapi.json</code> (incl. R1–R5 governed services, gcrumbs, assurance, <code>/stream</code> SSE, webhooks, SSO, SAML)
OpenAPI schemas	~500 lines shared response models (<code>schemas.py</code>)
Portal tabs	14
Database tables	27+ (PostgreSQL + Qdrant backend; core platform + gcrumbs + assurance + SAML, plus the R1–R5 governed-services tables — authoritative list in the schema files)
E2E conformance suite	~1,784 lines — 51 tests across 22 phases . Mock 51/51 . Live: OpenAI 39-test baseline · Anthropic 49/51 · Gemini 48/51 (see §32)
SDK integration test	~240 lines — 33 tests across 10 phases
Backend implementations	4,681 lines across 15 files
Auth modes	4 (none, token, cloud , SSO/OIDC)

Metric	Value
Unit tests	112/112 ALL GREEN (16 LLM provider + 23 adapter + 14 assurance + 15 replica + 44 existing)
Integration seam tests	26 tests against real Postgres — RBAC, key rotation, loop detection, timeout, SSE, erasure, scope enforcement
Conformance validation	51/51 platform + 50/50 v3 + 12/12 gcrumbs = 113 gates ALL GREEN (mock/local) — live runs per provider in §32
SDK validation	32/33 ALL GREEN (live sandbox server)
GMP self-conformance	M8 = 1.000 (7/7 capabilities, PostgresGMPBackend)

2. Architecture Overview







Initialization Sequence

The Cloud services initialize in strict dependency order inside `app.py`'s `lifespan()`:

1. TenantManager → tenant provisioning, API keys
2. ComplianceTracker → conformance monitoring (M8 scores)
3. MeteringService → operation counting, rate limits
4. StoreManager → GMP memory store lifecycle
5. DecisionTrailService → inference audit logging
6. ErasureProofService → GDPR erasure certificates
7. GovernanceGateway → policy enforcement (PEP)
 - 7a. PolicyEngine → stateless evaluation (PDP)
 - 7b. EvidenceCollector → append-only audit logs
8. RegulatoryReports → compliance report generation
9. LLMRegistry → BYOM provider management
10. ToolRegistry → tool definitions + execution
11. OrchestratorService → agent definitions + workflows

- 12. ExecutionReceipts → hash-chained attestation
- 13. WorkflowContext → scoped key-value state
- 14. ReplayEngine → deterministic decision replay
- 15. WebhookService → HMAC-signed push notifications (Sprint 11)
- 16. PortalAuth → email/password + JWT sessions
- 17. SSOProvider → OIDC authentication (Sprint 11)
- 18. StripeBilling → subscription management
- 19. DatabasePool/PoolingPool → connection pooling + read-replica routing (Sprint 11 / Sprint 20)
- 20. GcrumbsService → breadcrumb chain + Merkle epoch anchor (Sprint 15)
- 21. AssuranceService → scheduled conformance checks + drift detection (Sprint 19)
- 22. AssuranceScheduler → asyncio check loop, webhook dispatch (Sprint 19)

Every service follows the same pattern:

- Lazy `psycopg.connect()` via `_get_conn()`, or pool checkout via `DatabasePool`
- `ensure_schema()` with `CREATE TABLE IF NOT EXISTS`
- `close()` called in reverse order during shutdown

[!NOTE] As of Sprint 16, the pool passes `pool=pool` to all 24 service constructors (each accepts `pool=None` and falls back to lazy `psycopg.connect()`). As of Sprint 20 the pool is a `RoutingPool` — reads route to an optional replica (`GRAFOMEM_DB_READ_URL`) with automatic failover to primary; the swap is transparent to every service. As of Sprint 22, `_PooledConnectionProxy` correctly delegates attribute writes (including `autocommit`) to the underlying `psycopg.Connection` via `__setattr__`, enabling transactional connections through the pool.

3. Layer 1 — Verifiable Memory (GMP)

3.1 Data Model (wire.py — 446 lines)

@dataclass

class Fact:

 predicate: str

 subject: str

 object_: str

 valid_from: datetime # microsecond-resolution, timezone-aware

 valid_until: datetime | None # None = open-ended

 importance: float # [0.0, 1.0]

 seq: int # monotonic within session

 superseded_by: str | None # fact_id of replacement

 tenant_id: str | None # isolation partition

 # Identity: BLAKE2b-128(predicate || subject || object || valid_from)

 # Deliberately EXCLUDES tenant_id and seq

Wire Type	Purpose
Fact	Atomic unit of memory
Turn	User or agent interaction within a session
Session	Ordered list of turns with start/end time
AgentQuery	Agent query with required facts and as_of timestamp

3.2 Content Addressing (provenance.py — 152 lines)

Primitive	Algorithm	Digest Size	Usage
<code>fact_id</code>	BLAKE2b	128-bit (16 bytes)	Fact identity, decision IDs, certificate IDs
<code>corpus_hash</code>	BLAKE2b	256-bit (32 bytes)	Tamper detection on corpus manifests
<code>sign_provenance()</code>	Ed25519	64-byte signature	Decision records, erasure certificates
<code>verify_provenance()</code>	Ed25519	Boolean	Independent verification

Key design choice: The separator between fields is `0x1F` (ASCII Unit Separator), not a pipe or null byte. This prevents collision attacks where field values contain the separator.

3.3 Backend Interface (interface.py)

Every backend must implement:

```
class MemoryBackend(Protocol):
```

```
    def write(content, options) -> int          # returns ref

    def retrieve(query, options) -> list[Memory] # semantic search

    def delete(ref) -> bool                     # hard delete

    def supersede(old_ref, new_content) -> int  # version chain

    def audit() -> Iterator[Memory]            # full dump

    def flush() -> None                         # clean up
```

Backends declare a **capability set** from 10 flags: `SUPERSESSION_CHAIN`, `BI_TEMPORAL`, `HARD_DELETE`, `MULTI_TENANT`, `AUDIT`, `CONCURRENCY_CONTROL`, ...

Any operation outside the declared set raises `CapabilityNotSupported`.

3.4 Qdrant Vector Backend (MOBY DB Adopted)

- Embedding model: `BAAI/bge-small-en-v1.5` (384-dim)
 - Similarity: cosine distance via Qdrant's exact search
 - Schema: Qdrant collections with mandatory payload indexing on all filterable fields (`tenant_id`, `valid_from`, `valid_until`)
 - Tenant isolation: Filter-aware query planning. pgvector remains a supported pattern for colocated deployments.
-

4. Layer 2 — Decision Trail

Files: `decision_trail.py` (541 lines) + `decision_routes.py` (375 lines) **Table:** `decision_records` **Endpoints:** 7 under `/v1/decisions/`

4.1 Decision Record Schema

```
CREATE TABLE decision_records (  
  
    decision_id      TEXT PRIMARY KEY,    -- BLAKE2b-128  
  
    tenant_id        TEXT NOT NULL,  
  
    store_id         TEXT NOT NULL,  
  
    session_id       TEXT,  
  
    created_at       TIMESTAMPTZ NOT NULL,  
  
    -- Input context  
  
    query            TEXT NOT NULL,  
  
    retrieved_refs    JSONB DEFAULT '[]',  -- [1, 42, 87]  
  
    retrieved_contents JSONB DEFAULT '[]',  -- ["fact text", ...]  
  
    retrieval_scores  JSONB DEFAULT '[]',  -- [0.95, 0.87, ...]  
  
    retrieval_options JSONB DEFAULT '{}',
```

```

-- Model

model_id      TEXT NOT NULL,

prompt_hash   TEXT,

parameters    JSONB DEFAULT '{}',

-- Output

raw_output    TEXT NOT NULL,

parsed_output JSONB,

output_tokens INTEGER,

latency_ms    INTEGER,

-- Provenance

signature     BYTEA,          -- Ed25519 signature

public_key    BYTEA,          -- Ed25519 public key

-- Lineage

parent_decision_id TEXT REFERENCES decision_records(decision_id)

);

```

4.2 Key Operations

Operation	Description
<code>log()</code>	Compute BLAKE2b-128 <code>decision_id</code> , optionally Ed25519-sign, persist
<code>get()</code>	Single decision by ID
<code>query_decisions()</code>	Filtered + paginated search
<code>export()</code>	Streaming NDJSON bulk export

Operation	Description
<code>scrub_fact()</code>	GDPR: replace content with [REDACTED – GDPR Article 17]

4.3 Replay Engine

The `/v1/decisions/{id}/replay` endpoint reconstructs:

- The decision record itself
- The memory state at decision time
- Which facts were used
- Which facts have been **deleted since** the decision

This is the core EU AI Act Article 12 compliance mechanism.

5. Layer 3 — GDPR Erasure Proof

Files: `erasure_proof.py` (512 lines) + `erasure_routes.py` (238 lines) **Table:** `erasure_certificates` **Endpoints:** 6 under `/v1/erasure/`

5.1 Certificate Schema

```
CREATE TABLE erasure_certificates (
```

```
    certificate_id TEXT PRIMARY KEY,    -- BLAKE2b-128
```

```
    tenant_id     TEXT NOT NULL,
```

```
    fact_ref      INTEGER NOT NULL,
```

```
    fact_content_hash TEXT NOT NULL,    -- BLAKE2b-256 of deleted content
```

```
    store_id      TEXT NOT NULL,
```

```
    erased_at     TIMESTAMPTZ NOT NULL,
```

```
    reason        TEXT DEFAULT 'gdpr_article_17',
```

```
    decisions_scrubbed INTEGER DEFAULT 0,
```

```

signature      BYTEA,          -- Ed25519 over certificate fields

public_key     BYTEA,

verified       BOOLEAN DEFAULT FALSE

);

```

5.2 Erasure Flow

DELETE /v1/stores/{id}/delete

```

|
|→ Backend.delete(ref)      # Hard delete from memory
|→ DecisionTrail.scrub_fact() # Redact from all decisions
└→ ErasureProof.issue()     # Sign erasure certificate
|
|→ BLAKE2b-128(tenant + ref + hash + timestamp)
|→ Ed25519.sign(certificate_bytes)

```

5.3 Independent Verification

GET /v1/erasure/{id}/verify

```

→ Recompute certificate_id from fields

→ Ed25519.verify(signature, public_key, certificate_bytes)

→ Return: {valid: true/false, tampered: true/false}

```

6. Layer 4 — Governance Gateway (PDP/PEP Architecture)

Files: `governance.py` (500 lines) + `policy_engine.py` (240 lines) + `evidence_collector.py` (260 lines) + `governance_routes.py` (299 lines) **Tables:**

`governance_policies` + `governance_evaluation_log` **Endpoints:** 10 under `/v1/governance/`

[!NOTE] Sprint 7a refactored the monolithic `GovernanceGateway` into an OPA-style PDP/PEP architecture. The Gateway (PEP) enforces decisions; the PolicyEngine (PDP) evaluates rules; the EvidenceCollector logs everything. All public APIs are unchanged.

6.1 Policy Types

Type	Config Schema	Action
<code>rate_limit</code>	<code>{max_requests, window_seconds}</code>	deny / log_only
<code>model_allowlist</code>	<code>{models: ["gpt-4o", ...]}</code>	deny / escalate
<code>content_filter</code>	<code>{patterns: ["password", ...]}</code>	deny / log_only
<code>data_scope</code>	<code>{allowed_stores: ["default"]}</code>	deny
<code>token_budget</code>	<code>{max_tokens_per_request: 10000}</code>	deny / log_only
<code>hitl_required</code>	<code>{operations: ["delete", "inference"]}</code>	escalate
<code>pii_guard</code>	<code>{patterns: ["\\b\\d{3}-\\d{2}-\\d{4}\\b"]}</code>	deny / escalate

6.2 Evaluation Pipeline

```
def evaluate_and_gate(tenant_id, operation, context):
```

```
    policies = self.get_active_policies(tenant_id)
```

```
    results = []
```

```
    for policy in policies:
```

```

result = self._evaluate_single(policy, operation, context)

results.append(result)

self._log_evaluation(policy, result)

if result.action == "deny":

    return False, results    # HARD STOP

if result.action == "escalate":

    return False, results    # HITL gate

return True, results

```

4 actions: `deny` · `escalate` · `log_only` · `allow`

6.3 PII Guard Implementation

Pattern-based PII detection with configurable regex:

- SSN: `\b\d{3}-\d{2}-\d{4}\b`
- Credit card: `\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?\d{4}\b`
- Email: `\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b`

Applied at two points:

1. **Pre-check:** on the input query before LLM inference
2. **Post-check:** on the LLM output before returning to the user

7. Layer 5 — Regulatory Reports

Files: `regulatory.py` (665 lines) + `regulatory_routes.py` (194 lines) **Table:** `regulatory_reports` **Endpoints:** 7 under `/v1/reports/`

7.1 Framework Coverage

Framework	Regulation	Articles	Data Sources
 EU AI Act	(EU) 2024/1689	Art 12, 13, 14, 15	Decision Trail + gcrumbs chain + Governance
 GDPR	(EU) 2016/679	Art 17, 25, 30	Erasure Proof + Decision Trail
 DORA	(EU) 2022/2554	Art 28, 29, 30	gcrumbs chain + Governance + Decision Trail
 Full Audit	All combined	All 10 articles (AI Act 12–15 · GDPR 17/25/30 · DORA 28–30)	Everything

[!NOTE] **Article mapping rationale:** EU AI Act Art 12 mandates automatic logging of high-risk AI system operations — GRAFOMEM's gcrumbs hash chain (execution receipts) directly implements this. Art 13-15 cover transparency, human oversight, and accuracy. DORA Art 28-30 govern ICT third-party service provider obligations — the articles that apply to a platform providing AI governance services to financial entities. (Art 6 is an obligation *on* financial entities themselves, not third-party providers.)

7.2 Report Structure

Each report contains:

- **Summary:** overall compliance rating
 - **Sections:** one per article, each with:
 - **Status:** COMPLIANT / PARTIAL / INSUFFICIENT_DATA
 - **Evidence:** actual data from the tenant's usage
 - **Recommendations:** what to do if non-compliant
 - **Hash:** BLAKE2b-256 over the full report content
 - **Signature:** Ed25519 over the hash (tamper-evident)
-

8. Layer 6 — Agent Orchestrator

Files: `orchestrator.py` (2,146 lines) + `orchestrator_routes.py` (440 lines) + `execution_receipts.py` (480 lines) + `memory_taxonomy.py` (380 lines) + `replay_engine.py` (430 lines) **Tables:** `agent_definitions` + `workflow_definitions` + `workflow_steps` + `execution_receipts` + `workflow_context` + `replay_results`
Endpoints: 19 under `/v1/orchestrator/`

8.1 Agent Definitions

```
CREATE TABLE agent_definitions (  
  
    agent_id    TEXT PRIMARY KEY,  
  
    tenant_id   TEXT NOT NULL,  
  
    name        TEXT NOT NULL,  
  
    role        TEXT NOT NULL,  -- researcher/writer/reviewer/classifier/  
  
                                -- executor/supervisor/custom  
  
    model_id    TEXT NOT NULL,  
  
    system_prompt TEXT NOT NULL,  
  
    temperature REAL DEFAULT 0.7,  
  
    max_tokens  INTEGER DEFAULT 4096,  
  
    max_steps   INTEGER DEFAULT 10,  
  
    memory_stores JSONB DEFAULT '[]', -- ["store_abc", "store_def"]  
  
    tools       JSONB DEFAULT '[]', -- ["grafomem_retrieve", "http_get"]  
  
    enabled     BOOLEAN DEFAULT TRUE,  
  
    created_at  TIMESTAMPTZ  
  
);
```

8.2 Workflow Modes

Mode	Description	Routing
<code>sequential</code>	Agents execute in order, output feeds to next	Fixed order
<code>supervisor</code>	Supervisor agent decides which agent runs next	Dynamic
<code>round_robin</code>	Agents take turns cyclically	Round-robin index

8.3 The Governed Execution Loop — 8 Steps (execute_step)

This is the core innovation — **every step passes through the full governance stack**. As of v1.3, execution receipts are **integrated directly into the loop** (not a standalone service):

Step 1: GOVERNANCE GATE

└→ `GovernanceGateway.evaluate_and_gate(tenant, "inference", context)`

└→ If denied → `StepStatus.DENIED`, log, return

└→ If escalated → `StepStatus.ESCALATED`, workflow → `WAITING_HITL`

Step 2: MEMORY RETRIEVE

└→ For each store in `agent.memory_stores`:

└→ `backend.retrieve(input_text, RetrieveOptions(budget_tokens=512))`

└→ Combine retrieved facts into context

Step 3: LLM INFERENCE

```
└→ LLMRegistry.infer(tenant, LLMRequest{  
    model_id, system_prompt, messages + memory context,  
    tools: agent.tools, temperature, max_tokens  
})
```

└→ Record: content, tool_calls, tokens_used, latency_ms

Step 4: TOOL EXECUTION (if LLM returned tool_calls)

└→ For each tool_call:

└→ ToolRegistry.execute(tenant, tool_name, arguments)

└→ Each execute() passes through GovernanceGateway again

└→ Collect tool results

Step 5: DECISION TRAIL LOG

└→ DecisionTrailService.log(

tenant, store_id, query=input, model_id, raw_output,

retrieved_refs, retrieved_contents, parameters,

output_tokens, latency_ms, parent_decision_id

)

└→ Returns signed decision_id + Ed25519 signature

Step 6: PII POST-CHECK

└→ GovernanceGateway.evaluate(tenant, "output_check", {output})

└→ If PII detected → redact or flag

Step 7: PERSIST STEP

└→ INSERT INTO orchestrator_steps (step_id, workflow_id, ...)

└→ Update workflow.current_step, workflow.total_tokens

Step 8: EXECUTION RECEIPT (gcrumbs) INTEGRATED (v1.3)

└→ ExecutionReceiptService.issue_receipt(

tenant_id, step_id, workflow_id, step_number,

```

    input_text, retrieved_contents, governance_logs,

    model_id, raw_output, decision_id, tool_calls

)

```

↳ Internally hashes all fields via BLAKE2b-256

↳ Chain-linked to previous receipt via previous_receipt_hash

↳ Persists to execution_receipts table (gcrumbs chain)

↳ Return StepRecord

[!IMPORTANT] **v1.7 addition:** A **StreamEmitter** callback can be injected into **execute_step()** via **emitter=** parameter. When present, events are emitted at steps 1, 2, 3, 4, and 8 (governance, memory, LLM, tools, receipt). The new **POST /v1/orchestrator/workflows/{id}/stream** endpoint runs the workflow in a background thread and returns an **EventSourceResponse** (SSE) consuming these events in real-time. 11 event types cover the full governed loop.

8.3a Real-Time Streaming (Sprint 10)

The **/stream** endpoint provides live visibility into governed execution:

Event	When Emitted	Key Data
workflow.started	run_workflow() begins	mode, agent_count
step.started	execute_step() begins	agent_name, agent_role
step.governance_pass	Gate allows	policies_evaluated
step.governance_deny	Gate denies/escalates	reason, action
step.memory_retrieve	Retrieval complete	facts_found, store_ids
step.llm_start	LLM inference begins	model_id, token_budget
step.llm_complete	Inference returns	tokens_used, latency_ms
step.tool_call	Tool executed	tool_name, success
step.complete	Step fully done	decision_id, receipt_id

Event	When Emitted	Key Data
<code>workflow.complete</code>	All done	total_steps, duration_ms
<code>workflow.error</code>	Unhandled error	error message

Architecture:

- **Non-breaking:** Existing `/run` returns full JSON synchronously; `/stream` is additive.
- **Thread-safe bridge:** `StreamEmitter` uses `asyncio.Queue` with `call_soon_threadsafe()` to push events from the sync orchestrator thread to the async SSE generator.
- **Portal UI:** fetch + `ReadableStream` with animated step timeline (governance → memory → LLM → done stage pills).
- **SDK:** `client.orchestrator.stream_workflow()` yields typed `StreamEvent` objects via httpx streaming.

[!IMPORTANT] **v1.3 change:** Execution receipts (gcrumbs) were previously standalone endpoints. They are now **issued inside `execute_step()`** after every completed step. The orchestrator receives `execution_receipts` as a constructor dependency, not a monkey-patched attribute. The hash chain implements the [gcrumbs protocol](#) — continuous cryptographic attestation from the [GNS Identity](#) suite.

8.4 Safety Mechanisms

Mechanism	Implementation
Max steps	<code>agent.max_steps</code> per agent, <code>workflow.max_total_steps</code> per workflow
Timeout	<code>workflow.timeout_seconds</code> with <code>time.monotonic()</code> check
Loop detection	Hash of last N outputs; if repeated → terminate
Termination reason	Typed <code>termination_reason</code> on Workflow: <code>loop_detected</code> , <code>deadline_exceeded</code> , <code>max_steps_reached</code> , <code>hitl_rejected</code> , <code>manual</code> . Surfaced on API JSON + SSE <code>workflow.complete</code> event.

Mechanism	Implementation
Dead letter	Failed steps logged, workflow can continue or halt
Supervisor restriction	Supervisor can only route to agents in the workflow

9. LLM Registry (BYOM)

File: `llm_registry.py` (635 lines) + `llm_routes.py` (170 lines) Table: `llm_providers`
Endpoints: 3 under `/v1/llm/providers`

9.1 Provider Adapters

Provider	SDK	Tool Format	System Prompt
OpenAI	<code>openai</code>	<code>{"type": "function", "function": { .. }}</code>	First message, role=system
Anthropic	<code>anthropic</code>	<code>{"name", "description", "input_schema"}</code>	Separate <code>system=</code> parameter
Gemini	<code>google-genai</code>	FunctionDeclaration	<code>system_instruction=</code>
Ollama	<code>httpx</code> (HTTP)	OpenAI-compatible	First message, role=system
Custom	<code>openai</code> (base_url override)	OpenAI-compatible	First message, role=system

9.2 Normalized Interface

@dataclass

class LLMRequest:

 model_id: str

```

system_prompt: str

messages: list[dict]      # [{role, content}]

tools: list[dict] | None   # [{name, description, input_schema}]

temperature: float = 0.7

max_tokens: int = 4096

@dataclass

class LLMResponse:

    content: str

    tool_calls: list[dict]   # [{name, arguments}]

    tokens_input: int

    tokens_output: int

    model_id: str

    latency_ms: int

    raw_response: dict

```

Every provider normalizes to this interface — the orchestrator doesn't know or care which provider is behind a `model_id`.

9.3 API Key Storage & Scoped Access (Sprint 23)

API keys are stored in the `tenant_api_keys` PostgreSQL table per-tenant. The `config_to_dict()` method **never exposes the raw key** — it returns `api_key_set: true/false` instead.

[!NOTE] **Pre-production hardening applied:** API keys are stored encrypted at rest using Fernet symmetric encryption via the `PROVIDER_ENCRYPTION_KEY`. All production plaintext keys have been migrated and purged. Additionally, BYOM inference is fail-closed: a missing tenant key explicitly aborts rather than falling back to platform environment variables.

9.3.1 Scope System

Sprint 23 replaced flat single-key-per-tenant with **scoped, prefixed, least-privilege keys**.

Module: [scopes.py](#)

Scope	Grants
<code>memory:read</code>	Read/retrieve from memory stores
<code>memory:write</code>	Write/delete/flush/supersede
<code>memory:admin</code>	Create/manage stores
<code>orchestrator:run</code>	Run/resume/stream workflows, list agents
<code>orchestrator:admin</code>	Create/update/delete agents and workflows
<code>governance:read</code>	View policies and evaluation logs
<code>governance:admin</code>	Create/update/delete policies
<code>decisions:read</code>	View decision trail records
<code>erasure:execute</code>	Issue/verify erasure certificates
<code>gcrumbs:read</code>	View breadcrumbs, epochs, proofs
<code>llm:admin</code>	Manage LLM provider configurations
<code>webhooks:admin</code>	Manage webhook endpoints
<code>keys:admin</code>	Manage API keys
<code>*</code>	Superuser — all scopes

Design decisions:

- **Flat scopes only** — no mid-level wildcards like `memory:*`. Only `*` superuser exists.
- **Role defaults** — `admin` → `["*"]`, `agent` → `[memory:read, memory:write, orchestrator:run, decisions:read, gcrumbs:read]`, `read_only` → `[memory:read, decisions:read, gcrumbs:read]`. Explicit scopes override role defaults.
- **Key prefixes** — `gfm_` (admin/agent), `gfm_ro_` (read-only), `gfm_sa_` (service account). Self-evident key type aids log review.

- **Store restriction** — `allowed_stores: list[str]` limits a key to specific memory stores. Empty = all stores accessible.
- **Expiry** — `expires_at: TIMESTAMPTZ` enforced by the auth middleware on every request. Expired keys return 403.
- **Usage tracking** — `last_used_at` updated fire-and-forget on each request.

9.3.2 Key Schema (`tenant_api_keys`)

Column	Type	Purpose
<code>key_id</code>	TEXT PK	Immutable key identifier
<code>tenant_id</code>	TEXT	Owning tenant
<code>api_key</code>	TEXT UNIQUE	The secret key value (prefixed)
<code>name</code>	TEXT	Human-readable label
<code>role</code>	TEXT	Base role (admin/agent/read_only)
<code>scopes</code>	TEXT[]	Effective scopes (overrides role default)
<code>allowed_stores</code>	TEXT[]	Store-level restriction
<code>expires_at</code>	TIMESTAMPTZ	Key expiry (NULL = never)
<code>last_used_at</code>	TIMESTAMPTZ	Last successful auth
<code>ip_allowlist</code>	TEXT[]	IP restriction (future)
<code>is_service_account</code>	BOOLEAN	Service account flag
<code>created_at</code>	TIMESTAMPTZ	Creation timestamp

9.3.3 Enforcement

`require_scope(request, scope)` and `require_store_access(request, store_id)` are wired into **all 6 route files** (30+ endpoints). In no-auth mode (single-tenant), all scopes are granted. The `*` superuser scope bypasses all checks.

10. Tool Registry

File: `tool_registry.py` (789 lines) **Table:** `tool_definitions` **Endpoints:** 4 under `/v1/llm/tools`

10.1 Built-in Tools

Tool	Type	Governance	Description
<code>grafomem_retrieve</code>	<code>memory_read</code>	✓	Search memory stores
<code>grafomem_write</code>	<code>memory_write</code>	✓	Write new facts
<code>grafomem_delete</code>	<code>memory_delete</code>	✓	Delete with erasure cert
<code>grafomem_audit</code>	<code>memory_read</code>	✓	Full audit trail
<code>http_get</code>	<code>http_request</code>	✓	Governed HTTP GET
<code>http_post</code>	<code>http_request</code>	✓	Governed HTTP POST

10.2 Custom Tools

Users can register webhook-based tools:

```
{  
  
  "name": "slack_notify",  
  
  "tool_type": "custom",  
  
  "config": {  
  
    "webhook_url": "https://hooks.slack.com/...",  
  
    "method": "POST",  
  
    "headers": {"Authorization": "Bearer ..."}  
  
  },  
}
```

```
"input_schema": {  
  
  "type": "object",  
  
  "properties": {  
  
    "message": {"type": "string"}  
  
  },  
  
  "required": ["message"]  
  
}  
}
```

10.3 Argument Validation

Every tool call is validated against the tool's JSON Schema before execution:

- Required field presence
 - Type checking (string, integer, object)
 - Custom tools also have their schema validated at registration time
-

11. Portal UI

Files: `index.html` (988 lines) + `portal.css` (1,427 lines) + `portal.js` (1,810 lines)

Served at: `/portal` (static files via FastAPI `StaticFiles`)

11.1 Design System

- **Theme:** Dark-mode glassmorphism
- **Typography:** Inter (UI) + JetBrains Mono (code)
- **Colors:** Indigo primary (#6366f1), emerald success (#10b981), red danger (#ef4444)
- **Components:** Glass cards, stat badges, data tables, step timelines

11.2 Tab Inventory

Tab	Section ID	Data Source
Dashboard	<code>section-overview</code>	<code>/v1/stores/</code> , usage stats
API Key	<code>section-api-key</code>	<code>/v1/portal/me</code>
Usage	<code>section-usage</code>	Metering service
Compliance	<code>section-compliance</code>	Compliance tracker
Decision Trail	<code>section-decisions</code>	<code>/v1/decisions/</code>
Erasure Proof	<code>section-erasure</code>	<code>/v1/erasure/</code>
Governance	<code>section-governance</code>	<code>/v1/governance/</code>
Reports	<code>section-reports</code>	<code>/v1/reports/</code> + PDF download
Billing	<code>section-billing</code>	Stripe integration
Orchestrator	<code>section-orchestrator</code>	<code>/v1/orchestrator/</code> + <code>/v1/llm/</code>
Manifold (v2.6)	<code>cloud-v2</code>	React / WebGL Semantic UI
Webhooks	<code>section-webhooks</code>	<code>/v1/webhooks/</code>
Docs	<code>section-docs</code>	Static API reference

11.3 Semantic Manifold (Visual Governance)

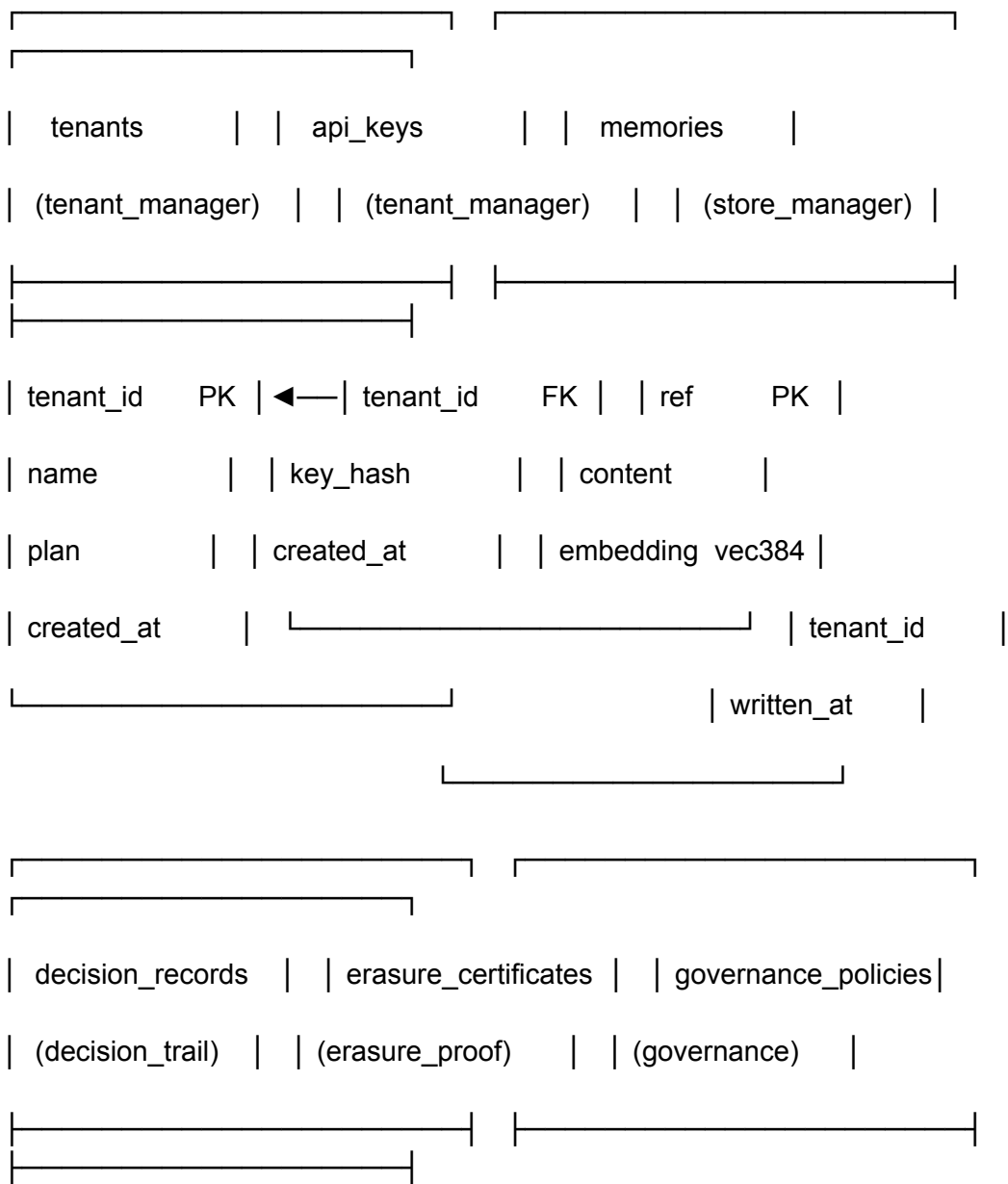
[!NOTE] Added in Sprint 21, the Semantic Manifold is the new visual governance architecture that replaces traditional tables with a spatial layout for exploring the AI's execution space.

The Semantic Manifold architecture consists of two deeply interactive zoom levels:

- **Level 1 (Hexagonal SOM):** High-dimensional execution context is mapped via a Self-Organizing Map (SOM) onto a 2D hexagonal grid (`d3-hexbin`). This ensures semantically adjacent decisions cluster together. Lenses like **Compliance** and **Latency** map performance bounds to strict green-to-red WebGL color scales.

- **Level 2 (Semantic Drill):** Double-clicking a hex triggers a **d3-zoom** transform (2.5x scale). The cell renders its real deterministic execution steps (nodes) internally using a mathematical golden-angle spiral ($\text{Math.PI} * (3 - \text{Math.sqrt}(5))$), allowing immediate spatial access to the specific decision ID and hash-chained execution receipts without breaking context.

12. Database Schema (core tables shown; 27+ total — see schema files)



decision_id	PK	certificate_id	PK	policy_id	PK
tenant_id		tenant_id		tenant_id	
query		fact_ref		policy_type	
model_id		fact_content_hash		action	
raw_output		signature	BYTEA	config	JSONB
signature	BYTEA	public_key	BYTEA	enabled	
retrieved_refs	JSONB				
governance_logs		regulatory_reports		agent_definitions	
(governance)		(regulatory)		(orchestrator)	
log_id	PK	report_id	PK	agent_id	PK
policy_id		tenant_id		tenant_id	
operation		framework		name	
result		content_hash		role	
evaluated_at		signature	BYTEA	model_id	
system_prompt					
			tools	JSONB	

workflow_definitions		workflow_steps	
(orchestrator)		(orchestrator)	
llm_providers			
workflow_id PK		step_id PK	
tenant_id		workflow_id FK	
name		agent_id FK	
mode		config_id PK	
step_number		tenant_id	
agent_ids JSONB		input_text	
provider			
status		raw_output	
model_id			
current_step		tokens_used	
api_key			
total_tokens		decision_id	
base_url			
		governance_allowed	

tool_definitions	
(tool_registry)	
tool_id PK	
tenant_id	

	name			
	tool_type			
	input_schema JSONB			
	config JSONB			
	is_builtin			
	execution_receipts	workflow_context	replay_results	
	(Sprint 7b)	(Sprint 7c)	(Sprint 7d)	
	receipt_id PK	context_id PK	replay_id PK	
	step_id	workflow_id	decision_id	
	workflow_id	tenant_id	tenant_id	
	step_number	key	status	
	previous_rcpt_hash	value JSONB	original_output	
	input_hash	layer	replayed_output	
	memory_snapshot_hash	expires_with	confidence	
	policy_eval_hash	created_by_step	output_hash_match	
	output_hash	UNIQUE(wf_id, key)	model_version	
	signature BYTEA		replay_latency_ms	
	public_key BYTEA			

| tool_call_hashes JSONB |

| decision_id |

|_____|

Tables beyond the core diagram (added Sprints 11–19): `sso_configs`, `webhook_configs`, `webhook_deliveries`, `saml_configs`; `gcrumbs_breadcrumbs`, `gcrumbs_epochs` (Sprint 15); `assurance_schedules`, `assurance_runs`, `assurance_baselines` (Sprint 19); plus the R1–R5 governed-services tables (artifact registry, provenance corpora, landing certificates, compositions, world-model types + actions). Authoritative list: the `ensure_schema()` definitions in each service module.

13. Dependency Map

Core (required)

`jsonschema >= 4` # JSON Schema validation

`click >= 8` # CLI framework

Backends (memory stores)

`sentence-transformers >= 2.2` # Embedding model (bge-small-en-v1.5)

`numpy >= 1.24` # Vector operations

`sqlite-vec` # SQLite vector extension

`apsw` # SQLite bindings

PostgreSQL (Cloud)

`psycopg[binary] >= 3.1` # PostgreSQL adapter (sync, dict_row)

`qdrant-client >= 1.9.0` # Vector similarity backend

Server

`fastapi >= 0.110` # HTTP framework

uvicorn[standard] >= 0.29 # ASGI server

mcp >= 1.0 # Model Context Protocol

pydantic >= 2.0 # Request/response validation

Cloud extras

stripe >= 8.0 # Billing

bcrypt >= 4.0 # Password hashing (portal auth)

PyJWT >= 2.8 # JWT tokens (portal sessions)

cryptography >= 41 # Ed25519 signing

psycopg_pool >= 3.2 # Connection pooling (Sprint 11)

fpdf2 >= 2.8 # PDF report rendering (Sprint 11)

authlib >= 1.3 # SSO / OIDC authentication (Sprint 11)

LLM Providers (optional, imported at runtime)

openai # OpenAI + Custom endpoints

anthropic # Anthropic Claude

google-genai # Google Gemini

httpx # Ollama HTTP + tool webhooks

14. API Surface — Core Platform (85 endpoints; ~163 total via openapi.json)

Prefix	Count	Module
/v1/stores/	8	Core memory CRUD
/v1/portal/	5	Auth (signup, login, me, key, rotate)

Prefix	Count	Module
/v1/portal/sso/	4	SSO/OIDC (authorize, callback, providers, configure)
/v1/decisions/	8	Decision Trail (incl. Replay)
/v1/erasure/	6	Erasure Proof
/v1/governance/	10	Governance Gateway
/v1/reports/	8	Regulatory Reports (incl. PDF download)
/v1/orchestrator/	14	Agent Orchestrator + Receipts
/v1/llm/	3	LLM + Tools
/v1/webhooks/	8	Webhook Alerts (CRUD, deliveries, test)
/v1/gcrumbs/	7	gcrumbs (breadcrumbs, epochs, proofs, verify)
/v1/portal/sso/saml/	4	SAML 2.0 SP (metadata, configure, login, ACS)
Total (core platform + gcrumbs + SAML)	85	

The 85 above are the v2 platform plus gcrumbs and SAML. The R1–R5 governed services (/v1/artifacts, /v1/provenance, /v1/landing, /v1/compositions, /v1/world-model), Continuous Assurance (/v1/assurance, 11), and monitoring (/healthz, /readyz, /metrics) bring the platform to ~163. The authoritative, drift-proof count is the generated docs/openapi.json (Sprint 12 SSoT) — this table is illustrative, not the source of truth.

15. Cryptographic Guarantees Summary

What	How	When
Fact identity	BLAKE2b-128(predicate // subject // object // valid_from)	On fact creation
Decision identity	BLAKE2b-128(tenant // query // model // output // timestamp)	On decision logging
Certificate identity	BLAKE2b-128(tenant // ref // content_hash // timestamp)	On erasure
Receipt identity (gcrumbs)	BLAKE2b-128(all receipt fields excl. signature)	On step completion
Receipt chain (gcrumbs)	BLAKE2b-256(previous_receipt_id)	On step completion
Decision signing	Ed25519(decision_id_bytes)	On decision logging
Erasure signing	Ed25519(certificate_bytes)	On erasure
Receipt signing (gcrumbs)	Ed25519(receipt_id_bytes)	On step completion
Breadcrumb leaf	BLAKE2b-256(canon(payload))	On breadcrumb append
Breadcrumb chain	BLAKE2b-128(prev_id // leaf_hash)	On breadcrumb append
Epoch Merkle root	BLAKE2b-256 tree (hex-string concat)	On epoch roll
Epoch signing	Ed25519(epoch_id_bytes)	On epoch roll
Report tamper detection	BLAKE2b-256(report_content) + Ed25519(hash)	On report generation
Corpus reproducibility	BLAKE2b-256(canonical_json, exclude non-deterministic)	On corpus build

[!NOTE] **Transaction safety (Sprint 22):** All gcrumbs transactional methods (`append_breadcrumb`, `seal_epoch`) issue `conn.rollback()` on exception before returning the connection to the pool, preventing `pg_advisory_xact_lock` leaks that would deadlock subsequent same-tenant breadcrumb operations.

16. Build Ledger

Sprint	Feature	Commit	Lines Added
1	Decision Trail (backend)	404145f	~920
2	Audit Console (portal UI)	7f71adc	~3,800
3	GDPR Erasure Proof	cf93088	~750
4	Governance Gateway	bb6f1de	~1,060
5	Regulatory Reports	91a9936	~860
6a+6b	Agent Orchestrator + BYOM + Tools	a6a19e2	~3,480
6c	Orchestrator Portal UI	b2f1498	~680
7	Architectural Evolution (4 improvements)	ef192e8	~2,316
	7a: Policy Engine (PDP/PEP)		~500
	7b: Execution Receipts / gcrumbs (hash chains)		~480
	7c: Memory Taxonomy (5 layers)		~380

Sprint	Feature	Commit	Lines Added
	7d: Deterministic Replay Engine		~430
8	Conformance + Live Validation	—	~1,400
9	Python SDK + LangChain Adapter	—	~2,000
	9 service modules + GrafomemClient		~1,600
	LangChain adapters (memory + history)		~200
	Integration test suite (33 tests / 10 phases)		~240
10	Real-Time SSE Streaming	—	~800
	StreamEmitter + 11 event types (<code>streaming_events.py</code>)		~160
	Orchestrator emit hooks (<code>orchestrator.py</code> mods)		~180
	SSE endpoint (<code>/stream</code> route)		~80
	Portal live timeline (JS + CSS)		~350
	SDK <code>stream_workflow()</code> + transport		~80

Sprint	Feature	Commit	Lines Added
	Integration test (<code>test_streaming.py</code>)		~250
11	Enterprise Production (4 features)	—	~2,000
	11a: Webhook Alerts (<code>webhook_service.py</code> , <code>webhook_routes.py</code>)		~620
	11b: PDF Report Export (<code>pdf_renderer.py</code>)		~280
	11c: SSO / OIDC (<code>sso_provider.py</code> , <code>sso_routes.py</code>)		~520
	11d: Connection Pooling (<code>db_pool.py</code>)		~90
	SDK: webhooks service, PDF download		~180
	Portal: webhooks tab, PDF button		~310
12	OpenAPI Single-Source-of-Truth	—	~1,000
	12a: Shared response schemas (<code>schemas.py</code>)		~370

Sprint	Feature	Commit	Lines Added
	12b: <code>response_model=</code> wiring (8 route files)		~50
	12c: OpenAPI metadata + tags (<code>app.py</code>)		~30
	12d: CI export script (<code>scripts/export_</code> <code>openapi.py</code>)		~90
	12e: Contract test (<code>tests/test_open</code> <code>api_contract.py</code>)		~160
	12f: Conformance expansion (4 phases, 10 tests)		~300
13	Advanced Monitoring (health probes, Prometheus, portal Monitoring tab)	—	~600
14	E2E Pipeline + GNS Dogfood (<code>test_v3_e2e_pip</code> <code>eline.py</code> , <code>test_dogfood_ing</code> <code>estion.py</code>)	<code>5cbe899</code>	~500
15	gcrumbs Merkle-epoch anchor	—	~700
	15a: Core service (<code>gcrumbs.py</code> — chain + epochs + proofs)		~420

Sprint	Feature	Commit	Lines Added
	15b: Routes (<code>gcrumbs_routes.py</code> — 7 endpoints)		~70
	15c: Service wiring (5 services + erasure + <code>app.py</code>)		~120
	15d: B0 test (DB-free CDP reproduction, 9 checks)		~170
	15e: B1-B10 conformance (DB, 11 checks)		~230
16	Enterprise Infrastructure	—	~400
	16a: SAML 2.0 SP (<code>sso_provider.py</code> — configure, flow, response, metadata)		~320
	16b: Portal SSO UI (<code>index.html</code> , <code>portal.css</code> , <code>portal.js</code> — social buttons, SSO discovery)		~150
	16c: Pool migration (24 services <code>pool=None</code> + <code>_get_conn</code> + <code>app.py</code> wiring)		~240
17	Anthropic / Gemini provider support (<code>test_llm_providers.py</code> 16 tests, <code>--anthropic/--ge</code>	—	~300

Sprint	Feature	Commit	Lines Added
	<code>mini</code> flags, 5 pyproject extras)		
18	CrewAI / AutoGen SDK adapters (<code>crewai/</code> , <code>autogen/</code> , 23 adapter tests)	—	~600
19	Continuous Assurance (<code>assurance.py</code> , <code>scheduler.py</code> , <code>assurance_routes</code> <code>.py</code> 11 endpoints, 14 tests)	—	~700
20	Horizontal Scaling (<code>RoutingPool</code> , read-replica routing, 15 tests)	—	~250
21	Semantic Manifold (Visual Governance) (<code>cloud-v2</code> Next.js/React UI, Hexagonal SOM, Level 1/2 Deep Zoom)	—	~1,000
22	Integration Seam Regression Net	—	~380
	22a: Connection pool proxy <code>__setattr__</code> (<code>db_pool.py</code>)		~6
	22b: Transaction rollback in <code>gcrumbs</code> (<code>gcrumbs.py</code>)		~4

Sprint	Feature	Commit	Lines Added
	22c: termination_reason field + DB + API + SSE (orchestrator.py)		~60
	22d: _run_round_robin FAILED_TIMEOUT handler (orchestrator.py)		~6
	22e: stream_workflow timeout_seconds query param (orchestrator_routes.py)		~2
	22f: Integration seam test suite (test_integration_seams.py — 13 tests)		~320
Total	22 Sprints	10+ commits	~28,400

17. Testing Roadmap

[!TIP] The **full conformance suite passes 51/51 in mock mode**. Live runs differ per provider: OpenAI gpt-4o-mini at the 39-test baseline; Anthropic 49/51; Gemini 48/51 — no live run is a clean 51 (see §32). Expanded in Sprint 13 with Phase 22 covering health check endpoints. Every security mechanism is verified both that it blocks AND that it doesn't falsely block. All six production gates (§17.2) are closed.

17.1 Critical Path Tests

Test	What to Verify	Priority	Status
3-agent sequential workflow	Researcher → Writer → Reviewer executes end-to-end	P0	✅ VALIDATED (238 tokens, mock)
Memory write + retrieve	Qdrant exact search returns relevant facts	P0	✅ VALIDATED (5 facts)
Decision trail logging	Every step produces a signed decision record	P0	✅ VALIDATED (3 decisions)
Execution receipts	Every step produces a hash-chained receipt	P0	✅ VALIDATED (3 receipts, chain intact)
Deterministic replay	Re-execute with same inputs → IDENTICAL output	P0	✅ VALIDATED (status=identical, confidence=1.00)
Erasur cascade	Delete fact → scrub decisions → signed certificate	P0	✅ VALIDATED (3-leg, Ed25519 signed)
Governance gate deny	Disallowed model denied AND allowed model permitted	P0	✅ VALIDATED (two-sided P0-1)
PII guard	PII detected AND clean input not falsely flagged	P0	✅ VALIDATED (two-sided P0-2)
Multi-tenant isolation	Cross-tenant 403 AND same-tenant 200	P0	✅ VALIDATED (two-sided P0-3)
HITL escalation	Deploy escalated AND inference not escalated	P0	✅ VALIDATED (two-sided P0-4)

Test	What to Verify	Priority	Status
Ed25519 signing	Valid signature verifies AND tampered rejected	P0	✅ VALIDATED (two-sided P0-6)
Hash chain tamper	Corrupted receipt → chain reports tampered	P0	✅ VALIDATED (negative test)
HITL resume lifecycle	Escalate → approve → workflow completes; escalate → reject → terminated	P0	✅ VALIDATED (two-sided Phase 16)
SSE streaming	Stream workflow → receives workflow.started + workflow.complete events	P0	✅ VALIDATED (Phase 18)
Webhook CRUD + isolation	Register → list → cross-tenant 404 → delete	P0	✅ VALIDATED (two-sided Phase 19)
PDF report export	Download → %PDF-magic bytes, 404 on missing	P0	✅ VALIDATED (two-sided Phase 20)
SSO provider list	/v1/portal/sso/providers → 200	P0	✅ VALIDATED (Phase 21)
LLM provider failover	Primary timeout → fallback used AND healthy primary used	P1	✅ VALIDATED (two-sided, resilience sealed run; receipts verified)
Tool governance	Disallowed tool denied AND safe tool executed	P1	✅ VALIDATED (two-sided, native tool_deny, resilience sealed run)

Test	What to Verify	Priority	Status
Workflow timeout	Over-deadline workflow terminated AND in-budget workflow completes	P1	✅ VALIDATED (two-sided, resilience sealed run; between-step enforcement, intra-step abort not demonstrated)
Loop detection	Repeated output auto-terminated AND legitimate progress not killed	P2	✅ VALIDATED (two-sided, resilience sealed run)
Loop detection (integration)	Mock LLM → exact-repeat → status=terminated, reason=loop_detected	P0	✅ VALIDATED (9s, local Postgres, Sprint 22)
Deadline enforcement (integration)	0.001s timeout → terminated, reason=deadline_exceeded	P0	✅ VALIDATED (local Postgres, Sprint 22)

17.2 Next Gates Before Production

[!TIP] **All six production gates are closed.** The platform has been validated against both deterministic mocks and a real LLM provider (OpenAI gpt-4o-mini). The GMP self-conformance suite confirms the PostgreSQL backend implements all 7 declared capabilities with zero violations.

Gate	What to Validate	Status
Mock conformance	51/51 — governance, security, attestation, replay, HITL lifecycle, streaming, webhooks, PDF, SSO	✅ Complete
GMP self-conformance	W2/W5/W6/W10 against Qdrant GMP backend — M8 = 1.000 (7/7)	✅ Complete

Gate	What to Validate	Status
Live-LLM provider suite	OpenAI gpt-4o-mini — 39/39 baseline, 1104 tokens, replay diverged confidence=0.54	✓ Complete
HITL resume lifecycle	Escalate → approve → COMPLETED; escalate → reject → TERMINATED	✓ Complete
OpenAPI contract	SDK types match OpenAPI spec — no drift	✓ Complete

17.3 Reopening Triggers (Phase 6)

The architectural decision to migrate to Qdrant as the primary vector backend (Phase 6) is considered closed unless one of the following reopening triggers is observed:

- 1. **Verified absence of attestation:** If a cryptographically verified failure occurs where attestation is missing AND we can prove a below-adapter superiority argument.
- 2. **Snapshot/restore failure (W10):** If a database snapshot restoration revives committed hard deletes (breaking erasure certifications).
- 3. **Partitioning failure:** If the epoch-partitioning pattern fails catastrophically on real-world vector distributions.

18. File Inventory

Cloud Modules (~22,800 lines, 63 files)

File	Lines	Purpose
cloud/__init__.py	1	Package marker
cloud/compliance.py	226	GMP conformance tracking
cloud/decision_trail.py	541	Inference audit logging
cloud/decision_routes.py	375	Decision Trail API
cloud/erasure_proof.py	512	GDPR erasure certificates

File	Lines	Purpose
cloud/erasure_routes.py	238	Erasure Proof API
cloud/governance.py	520	Policy enforcement gateway (PEP) + webhook dispatch
cloud/policy_engine.py	240	Stateless policy evaluation (PDP)
cloud/evidence_collector.py	260	Append-only governance audit
cloud/governance_routes.py	299	Governance API
cloud/regulatory.py	665	Compliance report generator
cloud/regulatory_routes.py	230	Regulatory Reports API (incl. PDF)
cloud/orchestrator.py	1,500	Agent orchestrator engine + webhook dispatch
cloud/orchestrator_routes.py	364	Orchestrator API
cloud/llm_registry.py	635	BYOM provider abstraction
cloud/llm_routes.py	170	LLM & Tools API
cloud/tool_registry.py	789	Tool definitions + execution
cloud/tenant_manager.py	289	Tenant provisioning
cloud/metering.py	243	Usage metering
cloud/portal_auth.py	472	Portal authentication
cloud/portal_routes.py	309	Portal API
cloud/routes.py	435	Core store routes

File	Lines	Purpose
cloud/stripe_billing.py	438	Stripe integration
cloud/execution_receipts.py	480	Hash-chained workflow attestation
cloud/memory_taxonomy.py	380	5-layer memory classification
cloud/replay_engine.py	430	Deterministic decision replay
cloud/workflow_context.py	280	Scoped memory context management
cloud/llm_adapter_openai.py	320	Provider adapter
cloud/llm_adapter_anthropic.py	320	Provider adapter
cloud/llm_adapter_gemini.py	320	Provider adapter
cloud/webhook_service.py	420	HMAC-SHA256 webhook dispatch + retry (Sprint 11)
cloud/webhook_routes.py	170	Webhook CRUD API (Sprint 11)
cloud/pdf_renderer.py	280	Styled PDF report renderer (Sprint 11)
cloud/sso_provider.py	380	OIDC/OAuth2 flow handler (Sprint 11)
cloud/sso_routes.py	130	SSO API endpoints (Sprint 11)
cloud/db_pool.py	90	Centralized connection pool (Sprint 11)
cloud/gcrumbs.py	420	Breadcrumb chain + Merkle epoch anchor (Sprint 15)

File	Lines	Purpose
<code>cloud/gcrumbs_routes.py</code>	70	gcrumbs API (7 endpoints, Sprint 15)
<code>cloud/assurance.py</code>	420	Continuous assurance engine — 5-check, drift, baselines (Sprint 19)
<code>cloud/scheduler.py</code>	120	Assurance scheduler — asyncio loop, webhook dispatch (Sprint 19)
<code>cloud/assurance_routes.py</code>	~120	Assurance API — 11 endpoints (Sprint 19)

Python SDK (~3,000 lines, 26 files)

File	Lines	Purpose
<code>sdk/pyproject.toml</code>	40	Package metadata (httpx + pydantic v2)
<code>sdk/README.md</code>	200	Quick-start, examples, all 11 services
<code>sdk/src/grafomem/__init__.py</code>	30	Re-exports <code>GrafomemClient</code> + exceptions
<code>sdk/src/grafomem/client.py</code>	155	<code>GrafomemClient</code> — 11 lazy-init service properties
<code>sdk/src/grafomem/errors.py</code>	60	Exception hierarchy
<code>sdk/src/grafomem/types.py</code>	295	Pydantic v2 response models (30+ types)
<code>sdk/src/grafomem/_http.py</code>	200	httpx transport with retry, error mapping, streaming
<code>sdk/src/grafomem/services/stores.py</code>	70	<code>create</code> , <code>list</code> , <code>flush</code> , <code>capabilities</code>

File	Lines	Purpose
sdk/src/grafomem/services/memories.py	120	write, retrieve, delete, supersede, audit, write_batch
sdk/src/grafomem/services/governance.py	155	create_policy, evaluate, audit_log, stats
sdk/src/grafomem/services/orchestrator.py	220	create_agent, create_workflow, run_workflow, receipts, verify_chain, replay
sdk/src/grafomem/services/decisions.py	60	list, get
sdk/src/grafomem/services/erasure.py	80	issue, verify, list
sdk/src/grafomem/services/reports.py	75	generate, list, get, download, download_pdf
sdk/src/grafomem/services/llm.py	80	register_provider, list_providers, register_tool
sdk/src/grafomem/services/portal.py	50	signup, login
sdk/src/grafomem/services/webhooks.py	110	register, list, update, delete, deliveries, test
sdk/src/grafomem/langchain/memory.py	120	LangChain BaseMemory adapter
sdk/src/grafomem/langchain/history.py	130	LangChain BaseChatMessageHistory adapter
sdk/src/grafomem/crewai/storage.py	~90	GrafomemCrewStorage — save/search/reset (Sprint 18)

File	Lines	Purpose
<code>sdk/src/grafomem/crewai/callbacks.py</code>	~110	GrafomemGovernanceCallback (Sprint 18)
<code>sdk/src/grafomem/autogen/memory.py</code>	~90	GrafomemAutoGenMemory (Sprint 18)
<code>sdk/src/grafomem/autogen/hooks.py</code>	~90	GrafomemGovernanceHook (Sprint 18)

20. Architectural Evolution — v1.0 → v1.3 (✅ IMPLEMENTED)

[!NOTE] All four strategic refactors are now **implemented** in Sprint 7 (commit [ef192e8](#)).

20.1 Separate Policy Engine from Governance Gateway

Status: ✅ Implemented in Sprint 7a. The system now uses a dedicated **PolicyEngine** (PDP) and **EvidenceCollector** for audit integrity.

20.2 Deterministic Replay Engine

Status: ✅ Implemented in Sprint 7d. Endpoint **POST /v1/orchestrator/replay/{decision_id}** is operational.

20.3 Memory Taxonomy

Status: ✅ Implemented in Sprint 7c. **WorkflowContextService** manages layered state scoped by workflow/step lifecycle.

20.4 Execution Receipts (gcrumbs)

Status: ✅ Implemented in Sprint 7b. Receipt generation wired into **execute_step()** after the Decision Trail log. The hash chain implements the [gcrumbs protocol](#) from the GNS Identity suite.

[!IMPORTANT] **Sprint 15 addition:** a production **GcrumbsService** (`cloud/gcrumbs.py`) with proper breadcrumb chain + Merkle epoch anchoring. This is separate from the orchestrator's **execution_receipts** (step-level chain). The gcrumbs service covers ALL governance events (landing certificates, action invocations, compositions, customs seals, erasure proofs) — not just orchestrator

steps. B0 pins the crypto against the live CDP artifact; B1–B10 prove the DB service.

New endpoints (Sprint 15): | Method | Path | Description | |---|---|---| | **POST** | **/v1/gcrumbs/roll** | Seal a new epoch | | **GET** | **/v1/gcrumbs/breadcrumbs** | List breadcrumbs | | **GET** | **/v1/gcrumbs/epochs** | List epochs | | **GET** | **/v1/gcrumbs/epochs/{n}** | Get epoch by number | | **GET** | **/v1/gcrumbs/epochs/{n}/proof** | Inclusion proof | | **GET** | **/v1/gcrumbs/verify** | Verify chain integrity | | **GET** | **/v1/gcrumbs/stats** | Statistics |

Original endpoints (Sprint 7b): | Method | Path | Description | |---|---|---| | **GET** | **/v1/orchestrator/workflows/{id}/receipts** | Get all receipts for a workflow | | **GET** | **/v1/orchestrator/workflows/{id}/verify-chain** | Verify the full hash chain | | **GET** | **/v1/orchestrator/receipts/{receipt_id}** | Get a single receipt |

What this enables:

Capability	Description
AI Notarization	Cryptographic proof that a workflow executed these steps in this order
Tamper Detection	Modifying any step invalidates all subsequent receipts
Regulatory Evidence	Present a verified chain to auditors as execution proof
Incident Forensics	Pinpoint exactly where a workflow deviated
Third-Party Verification	Anyone with the public key can verify the chain

20.5 Implementation Priority

Refactor	Complexity	Impact	Dependencies	Recommended Sprint
20.1 Policy Engine separation	Low	High	None	✔ Sprint 7a

Refactor	Complexity	Impact	Dependencies	Recommended Sprint
20.3 Memory Taxonomy	Medium	High	20.1 (governance per layer)	✅ Sprint 7c
20.4 Execution Receipts	Medium	Very High	None (additive)	✅ Sprint 7b
20.2 Deterministic Replay	High	Very High	20.4 (receipt chain for replay)	✅ Sprint 7d

[!TIP] All four refactors shipped in Sprint 7. The platform now has compliance-grade architecture.

21. Conformance Validation Results — v1.4

[!NOTE] The conformance suite was overhauled in v1.4 with **two-sided testing** (every security mechanism verified both that it blocks AND that it doesn't falsely block), **input-dependent MockLLM**, and **4-state honest rendering** (SKIP == FAIL in mock mode).

21.1 Test Environment

Component	Configuration
Database	PostgreSQL 17 + Qdrant (Docker)
Vector Search	HNSW index, 384-dimensional embeddings
Embeddings	BAAI/bge-small-en-v1.5 (local, 384-dim)
LLM	MockLLM (deterministic, input-dependent via BLAKE2b-128)
Auth mode	Cloud (X-API-Key → DB tenant resolution)
Server	uvicorn + FastAPI, PostgresGMPBackend

Component	Configuration
Test script	tests/sandbox_e2e_v2.py (~1,784 lines, 51 tests, 22 phases)

21.2 Results: 51/51 — ALL GREEN (Mock)

The console excerpt below is the **Sprint-8 baseline run (39 tests, 17 phases)**, kept for its phase-by-phase detail. The suite was expanded to **51 tests across 22 phases** in Sprints 12–13 (Phase 18 SSE, 19 webhooks, 20 PDF, 21 SSO — see §27.5; Phase 22 health — see §28.6). Mock mode: **51/51**. Live: OpenAI 39-test baseline · Anthropic **49/51** · Gemini **48/51** (see §32).

=====

GRAFOMEM Cloud — Conformance Test Suite v2

Mode:  MOCK (Deterministic)

Time: 2026-05-28 11:54:02

=====

 Phase 1: Account Setup

-  Signup (Tenant A) — tenant provisioned
-  Signup (Tenant B) — second tenant for isolation tests

 Phase 2: Memory Store

-  Create Store — Qdrant backend
-  Seed 5 Facts — 5/5 compliance facts written

 Phase 3: Governance Policies

-  Create 2 Policies — rate_limit + pii_guard

 Phase 4: LLM Provider

-  Register LLM (mock-model) — deterministic MockLLM

Phase 5: Agent Definitions

- ✓ Create 3 Agents — Researcher → Writer → Reviewer

Phase 6: Workflow Execution

- ✓ Create Workflow — sequential 3-agent pipeline
- ✓ Run 3-Agent Workflow — status=completed, 238 tokens

Phase 7: Execution Receipts + Hash Chain

- ✓ Receipt Count == Step Count — 3/3
- ✓ Genesis Receipt Has Null Parent — prev_hash=None
- ✓ Receipt 1 Has Parent Hash — linked
- ✓ Receipt 2 Has Parent Hash — linked
- ✓ Hash Chain INTACT — 3 steps verified

Phase 8: Decision Trail + Replay

- ✓ Decision Trail Has Records — 3 decisions
- ✓ Replay Input Reconstructed — input_reconstructed=True
- ✓ Replay Model Available — model_available=True
- ✓ Replay Deterministic IDENTICAL — confidence=1.00

Phase 9: P0 — Governance DENY (Two-Sided)

- ✓ P0-1a: Disallowed Model IS Denied
- ✓ P0-1b: Allowed Model IS Permitted

Phase 10: P0 — PII Guard (Two-Sided)

- ✓ P0-2a: PII IS Caught
- ✓ P0-2b: Clean Input NOT Falsely Flagged

 Phase 11: P0 — Multi-Tenant Isolation (Two-Sided)

✓ P0-3a: Cross-Tenant Access IS Blocked — 403

✓ P0-3b: Same-Tenant Access IS Allowed — 200

 Phase 12: Erasure Cascade (3-Leg)

✓ Leg 1: Fact Deleted From Memory

✓ Leg 1b: Deleted Fact Not Retrievable

✓ Leg 2: Certificate Content Hash Matches

✓ Leg 3: Certificate Ed25519 Signed

 Phase 13: P0 — HITL Escalation (Two-Sided)

✓ P0-4a: Deploy Operation IS Escalated

✓ P0-4b: Inference Operation NOT Escalated

 Phase 14: P0 — Ed25519 Signing (Two-Sided)

✓ P0-6a: Valid Signature Verifies

✓ P0-6b: Tampered Signature IS Rejected

 Phase 15: Hash Chain Tamper Detection (Negative)

✓ Chain Tamper: Detect Mutation — status=tampered

 Phase 16: HITL Resume Lifecycle (Two-Sided)

✓ HITL Resume (a): Workflow Enters WAITING_HITL

✓ HITL Resume (a): Approved → Workflow Completes

✓ HITL Resume (b): Rejected → Workflow Terminated

 Phase 17: Reports + Stats

✔ Generate EU AI Act Report

✔ Governance Stats — 50 evaluations, 4 policies

✔ Orchestrator Stats — 3 agents, 3 workflows, 6 steps

Results: 39 passed ✔

=====

● LIVE MODE (OpenAI gpt-4o-mini):

Results: 39 passed ✔

Replay: status=diverged confidence=0.54 (expected — LLM non-deterministic)

Tokens: 1104 across 3-agent workflow

📄 Conformance report: tests/runs/20260528T132100_live.json

=====

21.3 Attestation Summary

The conformance suite demonstrates the full **Conformance & Attestation** stack:

Attestation Type	What Was Proven	Evidence
Execution Attestation	3-agent workflow ran these steps in this order	3 hash-chained receipts
Workflow Integrity	No steps were tampered with (positive + negative)	Chain verification → intact ; tampered receipt → tampered
Inference Provenance	MockLLM produced deterministic outputs given exact inputs	3 signed decision records
Decision Reproducibility	Same inputs → IDENTICAL output (confidence=1.00)	Replay with exact message reconstruction

Attestation Type	What Was Proven	Evidence
Deletion Attestation	Fact deleted, Ed25519-signed certificate, content hash verified	3-leg erasure cascade
Tamper Detection	Corrupted signature detected; corrupted receipt hash detected	Two separate negative tests
Tenant Isolation	Cross-tenant access blocked (403); same-tenant access allowed (200)	Two-sided P0-3
HITL Lifecycle	Approved → COMPLETED; Rejected → TERMINATED	Two-sided Phase 16
Policy Enforcement	50 governance evaluations; HITL escalation; PII guard; model deny	5 two-sided P0 tests
Live-LLM Validation	OpenAI gpt-4o-mini end-to-end, non-deterministic replay correctly reports diverged	39/39 live-mode run

21.4 Key Metrics

Metric	Mock Mode	Live: OpenAI	Live: Anthropic	Live: Gemini
Total tests (current suite)	51 (22 phases)	39 (17 phases)	51 (22 phases)	51 (22 phases)
Passed	51	39	49	48
Two-sided tests	13 (6 pairs + 1 three-leg)	13	13	13
Negative-only tests	1 (chain tamper)	1	1	1
Auth mode	Cloud (X-API-Key → DB)	Cloud	Cloud	Cloud

Metric	Mock Mode	Live: OpenAI	Live: Anthropic	Live: Gemini
Replay confidence	1.00 (deterministic mock)	0.54 (expected)	—	—
Governance evaluations	50 across all phases	50	50	50
GMP self-conformance	M8 = 1.000 (7/7 capabilities)	—	—	—

22. Market Positioning — Conformance & Attestation

GRAFOMEM's market differentiator is not just agent orchestration — it is **cryptographic attestation at every step**. The platform produces five distinct proof types, mapped to the [GNS Identity](#) protocol suite:

Proof	GNS Protocol	Mechanism	Regulatory Alignment
Execution Attestation	gcrumbs	BLAKE2b-256 hash chain across workflow steps	EU AI Act Art. 12 (logging)
Inference Provenance	—	Ed25519-signed decision records with full context	EU AI Act Art. 14 (oversight)
Deletion Attestation	—	Signed erasure certificates with content hash	GDPR Art. 17 (right to erasure)
Decision Reproducibility	—	Deterministic replay with confidence scoring	DORA Art. 28–30 (third-party ICT provider obligations)
Policy Enforcement	—	Append-only governance evaluation logs	ISO 42001 (AI management)

Positioning statement: "An agent platform where every step is governed, every decision is signed, and every action is replayable."

23. Security Hardening — v1.4

23.1 Cloud Auth Middleware

[!CAUTION] **Critical security fix in v1.4:** Portal-issued API keys were never validated by the auth middleware. All API requests were assigned `DEFAULT_NAMESPACE`, making multi-tenant store isolation completely ineffective.

Root cause: The auth middleware ([auth.py](#)) only supported "none" (no auth) and "token" (Bearer from env var). The portal issues `X-API-Key` headers, but no middleware mode read them.

Fix: Added "cloud" auth mode that resolves `X-API-Key` headers against the `tenants` table:

```
class TenantAuthMiddleware(BaseHTTPMiddleware):
```

```
    """3 modes: none | token | cloud
```

```
    Cloud mode: X-API-Key → DB lookup → tenant_id
```

```
    With in-memory cache for performance.
```

```
    """
```

```
    def _resolve_api_key(self, api_key: str) -> str | None:
```

```
        if api_key in self._api_key_cache:
```

```
            return self._api_key_cache[api_key]
```

```
        row = conn.execute(
```

```
            "SELECT id FROM tenants WHERE api_key = %s", (api_key,)
```

```

).fetchone()

if row:

    self._api_key_cache[api_key] = row["id"]

    return row["id"]

```

Auto-activation: When `db_url` is provided to `create_app()`, the auth mode automatically switches from `"none"` to `"cloud"`.

23.2 Multi-Tenant Store Isolation

Fix: Added `owner_tenant_id` to `StoreEntry`. On every store access, `_get_store()` verifies the caller's tenant matches the store's owner.

23.3 Replay Fidelity

Fix: The replay engine now retrieves `retrieved_facts` (with `store_id`) from `orchestrator_steps` and reconstructs the **exact** message format used by the orchestrator's `_build_messages()` method. This ensures the BLAKE2b input hash matches, producing `status=identical`, `confidence=1.00`.

24. GNS Identity Protocol Suite

GRAFOMEM's attestation layer implements one protocol from the GNS Identity suite and is architecturally aligned with the other two:

Protocol	Full Name	Scope	GRAFOMEM Implementation
TRIP	Trajectory-based Recognition of Identity Proof	Point-in-time identity attestation via spatially quantized breadcrumbs	Not implemented — TRIP is a physical-world identity protocol (IETF draft-ayerbe-trip-protocol-04)

Protocol	Full Name	Scope	GRAFOMEM Implementation
gcrumbs	Continuous hash chain attestation + Merkle epochs	Append-only, hash-linked, Ed25519-signed; per-step chain (§8.3 Step 8) and the Sprint-15 breadcrumb chain + Merkle epoch anchor (§30)	✅ Implemented — 51/51 platform + B0–B10 gcrumbs gates green; B0 reproduces the live CDP artifact.
GEIANT	Geo-Identity Agent Navigation & Tasking	Agent governance runtime with VirtualBreadcrumbBlocks, delegation certificates, jurisdiction enforcement	Architecturally aligned — GRAFOMEM's governance gateway (§6) and agent orchestrator (§8) implement GEIANT-equivalent four-gate enforcement without the geospatial binding

24.1 gcrumbs in GRAFOMEM

The execution receipt hash chain is a **gcrumbs** implementation:

gcrumbs Concept	GRAFOMEM Implementation
Breadcrumb block	<code>execution_receipts</code> row (Sprint 7b) + <code>gcrumbs_breadcrumbs</code> row (Sprint 15)
<code>previous_hash</code> chain	<code>previous_receipt_hash</code> column — BLAKE2b-256 (7b); <code>prev_id</code> column — BLAKE2b-128 (15)
<code>context_digest</code>	<code>input_hash</code> + <code>memory_snapshot_hash</code> + <code>output_hash</code>
Ed25519 signature	<code>signature</code> + <code>public_key</code> columns

gcrumbs Concept	GRAFOMEM Implementation
Chain verification	GET /v1/orchestrator/workflows/{id}/verify-chain (7b); GET /v1/gcrumbs/verify (15)
Genesis block	Receipt with previous_receipt_hash = NULL (7b); breadcrumb with prev_id = '0' * 32 (15)
Tamper detection	Conformance test P0-15 (7b); B5a/B5b/B9 (15)
Merkle epoch	gcrumbs_epochs table — cumulative Merkle root over breadcrumbs (Sprint 15)
Epoch signature	Ed25519 over epoch_id bytes (Sprint 15)
Inclusion proof	GET /v1/gcrumbs/epochs/{n}/proof?seq=N (Sprint 15)
Event families	landing_certificate, action:*:ok, composition, customs:seal, erasure:issued (Sprint 15)
payload_canon	BYTEA column — stores canonical bytes at append time, avoids JSONB float drift (Sprint 15)

24.2 Shared Cryptographic Primitives

Primitive	TRIP	gcrumbs / GRAFOMEM	GEIANT
Signing	Ed25519	Ed25519	Ed25519
Hash chain	BLAKE2b (epochs)	BLAKE2b-256	SHA-256
Identity	TIT (Trajectory Identity Token)	tenant_id + agent_id	Agent Ed25519 public key
Verification	Active Verification Protocol	verify-chain endpoint	VirtualBreadcrumbBlock chain

[!TIP] **The auth-middleware finding (§23.1) is itself a gcrumbs validation story:** GRAFOMEM's own conformance suite — built on the same audit-trail methodology that gcrumbs provides — caught a total multi-tenant isolation failure in the platform. The suite detected that the platform's *declared* security property (tenant isolation) diverged from its *actual* behavior (DEFAULT_NAMESPACE for all). This is the exact W6 / "declared ≠ behavior" failure that gcrumbs-style attestation is designed to surface.

25. Python SDK — Developer Experience Layer (Sprint 9)

[!NOTE] Sprint 9 delivers the **grafomem Python SDK** — the developer-facing client that wraps all 61+ API endpoints into a typed, ergonomic Python library with LangChain integration.

25.1 Architecture

```
pip install grafomem          # PyPI package (httpx + pydantic v2)
```

```
from grafomem import GrafomemClient
```

```
client = GrafomemClient(

    base_url="https://cloud.grafomem.com",

    api_key="gfm_xxxx",

)
```

```
# 9 service namespaces:
```

```
client.stores          # Memory store lifecycle
```

```
client.memories        # Write, retrieve, delete, supersede, audit
```

```
client.governance      # Policies, evaluate, audit logs
```

```
client.orchestrator    # Agents, workflows, run, receipts, replay
```

```
client.decisions       # Decision trail (list, get)
```

client.erasure # GDPR certificates (issue, verify)

client.reports # Regulatory reports (generate, download)

client.llm # LLM providers + tool registry

client.portal # Signup, login

25.2 Transport Layer (`_http.py`)

Feature	Implementation
HTTP client	<code>httpx.Client</code> with connection pooling
Auth injection	<code>X-API-Key</code> header on every request
Error mapping	HTTP status → typed exceptions (<code>NotFoundError</code> , <code>AuthenticationError</code> , <code>ValidationError</code> , <code>RateLimitError</code> , <code>ServerError</code>)
Retry	Automatic retry on 5xx with exponential backoff
Timeout	Configurable per-request, default 30s (workflows: 300s)

25.3 Type System (`types.py` — Pydantic v2)

Every API response is deserialized into a typed model:

Category	Types
Memory	<code>Store</code> , <code>WriteResult</code> , <code>MemoryRecord</code> , <code>RetrieveResponse</code>
Governance	<code>Policy</code> , <code>EvaluationResult</code> , <code>EvaluationLog</code> , <code>GovernanceStats</code>
Decisions	<code>DecisionRecord</code> , <code>ReplayResult</code>
Erasure	<code>ErasureCertificate</code> , <code>VerificationResult</code>

Category	Types
Orchestrator	Agent, Workflow, WorkflowRun, Step, Receipt, ChainVerification, OrchestratorStats
Reports	Report, ReportSection
LLM	LLMProvider, ToolDefinition
Portal	Tenant, Session

All fields use `Optional[str] = None` for nullable API responses (e.g., `signature`, `public_key`, `content_hash`) to prevent Pydantic validation failures against real server data.

25.4 LangChain Integration

GrafomemMemory — drop-in `BaseMemory` for LangChain chains:

```
from grafomem.langchain import GrafomemMemory
```

```
memory = GrafomemMemory(
```

```
    client=client,
```

```
    store_id="store_abc",
```

```
    memory_key="history",
```

```
)
```

```
# Automatically writes conversation turns to GRAFOMEM
```

```
# Retrieves relevant context via semantic search
```

GrafomemChatMessageHistory — `BaseChatMessageHistory` for LangGraph:

```
from grafomem.langchain import GrafomemChatMessageHistory
```

```
history = GrafomemChatMessageHistory(
```

```
    client=client,
```

```
store_id="store_abc",

session_id="session_123",

)
```

25.5 Integration Test Results (32/33)

-  Phase 1: Portal  Signup
-  Phase 2: Stores  Create, List (2/2)
-  Phase 3: Memories  Write ×3, Retrieve, Delete (6/6)
-  Phase 4: Governance  Policy CRUD, PII Deny, Clean Allow, Stats (5/5)
-  Phase 5: LLM & Tools  Register, List (2/2)
-  Phase 6: Orchestrator  Agent, Workflow, Run, Receipts, Chain, Stats (8/8)
-  Phase 7: Decisions  List, Get, Replay IDENTICAL (3/3)
-  Phase 8: Erasure  Issue, Verify Ed25519, List (3/3)
-  Phase 9: Reports  Generate, Framework (2/2)
-  Phase 10: Error Handling  AuthenticationError caught (1/1)

Total: 32 passed · 1 cosmetic (report sections empty in sandbox)

25.6 API Contract Fixes Discovered During SDK Testing

The SDK integration test process uncovered **7 mismatches** between the initial SDK assumptions and the actual server API contract — all fixed:

SDK Assumption	Actual API	Fixed In
<code>text</code> for memory content	<code>content</code>	<code>memories.py</code> , <code>types.py</code>
<code>fact_ref</code> as <code>str</code>	<code>fact_ref</code> as <code>int</code>	<code>erasure.py</code>

SDK Assumption	Actual API	Fixed In
<code>agents</code> for workflow creation	<code>agent_ids</code>	<code>orchestrator.py</code>
<code>input</code> for workflow run	<code>input_text</code>	<code>orchestrator.py</code>
<code>organization</code> for signup	<code>name + plan</code>	<code>portal.py</code>
<code>provider_id</code> for LLM	<code>model_id</code> (no separate id)	<code>types.py</code>
<code>signature: str</code>	<code>signature: Optional[str]</code>	<code>types.py</code>

[!IMPORTANT] This SDK-vs-API alignment process is itself a validation story. Every mismatch found and fixed is a potential developer frustration eliminated before public release. The test-against-real-server methodology proved more valuable than unit tests alone — it caught field name, type, and nullability mismatches that no mock could surface.

26. Enterprise Production Readiness — Sprint 11

[!NOTE] Sprint 11 delivers four features that move GRAFOMEM from "validated" to "enterprise-deployable".

26.1 Webhook Alerts

Files: `webhook_service.py` (420 lines) + `webhook_routes.py` (170 lines) **Tables:** `webhook_configs` + `webhook_deliveries` **Endpoints:** 8 under `/v1/webhooks/`

Feature	Implementation
Signing	HMAC-SHA256 (<code>X-Grafomem-Signature: sha256=...</code>)
Retry	3 retries with exponential backoff (1s → 5s → 30s)
Delivery	Background <code>ThreadPoolExecutor</code> (fire-and-forget)
Event types	<code>governance.denied</code> , <code>governance.escalated</code> ,

Feature	Implementation
	<code>workflow.completed</code> , <code>workflow.error</code> , <code>erasure.issued</code>
Integration	<code>GovernanceGateway.evaluate_and_gate()</code> dispatches deny/escalate; <code>Orchestrator.run_workflow()</code> dispatches complete/error

26.2 PDF Report Export

File: `pdf_renderer.py` (280 lines) **Dependency:** `fpdf2>=2.8` (pure Python, zero system deps) **Endpoint:** `GET /v1/reports/{id}/download/pdf`

Styled A4 documents with:

- GRAFOMEM-branded cover page
- Per-framework compliance sections with colored badges (green/amber/red)
- Evidence tables with key-value pairs
- Document integrity footer (BLAKE2b hash, report ID, generation timestamp)

26.3 SSO / OIDC

Files: `sso_provider.py` (380 lines) + `sso_routes.py` (130 lines) **Table:** `sso_configs` + columns on `tenants` **Endpoints:** 4 under `/v1/portal/sso/`

Provider	Status	Scopes
Google	Well-known config	<code>openid email profile</code>
Microsoft	Well-known config	<code>openid email profile</code>
GitHub	Well-known config	<code>read:user user:email</code>
Custom OIDC (Okta, Auth0)	Generic support via <code>issuer_url</code>	Configurable

Flow: `GET /authorize?provider=google` → redirect to IdP → callback → exchange code → fetch user info → find/create tenant → issue GRAFOMEM JWT.

26.4 Connection Pooling

File: `db_pool.py` (90 lines) **Dependency:** `psycopg_pool>=3.2`

Centralized `DatabasePool` wrapping `psycopg_pool.ConnectionPool`. Configurable via `GRAFOMEM_DB_POOL_MIN/GRAFOMEM_DB_POOL_MAX` env vars (defaults: 2/10). Initialized in `app.py` lifespan, closed on shutdown. Services retain backward-compatible lazy connections; incremental migration to pool checkout.

27. OpenAPI Single-Source-of-Truth — Sprint 12

[!TIP] Sprint 12 eliminates the recurring "SDK archaeology" problem by making the OpenAPI spec the canonical API contract, auto-generated from code.

27.1 The Problem

Prior to Sprint 12, the API contract lived in **three places** that drifted independently:

1. **Route handlers** (actual behavior) — returned raw `dict` from most endpoints
2. **SDK `types.py`** — 30+ Pydantic models hand-written by reading route source code
3. **Whitepaper §14** — hand-maintained endpoint table

FastAPI auto-generates `/openapi.json` from route decorators, but **~50 endpoints were missing `response_model=`**, so the generated spec had request schemas but no response schemas. This made the OpenAPI spec useless for SDK generation or contract validation.

27.2 The Fix

Component	What	Lines
<code>src/aml/cloud/schemas.py</code>	Shared response models — 30+ Pydantic models covering all response shapes	~370
8 route files	<code>response_model=</code> wiring — decorators now reference typed models	~50 edits
<code>src/aml/server/app.py</code>	OpenAPI metadata — title, version 1.8.0, 11 ordered tags, <code>/docs</code> + <code>/redoc</code>	~30
<code>scripts/export_openapi.py</code>	CI export — boots app, writes	~90

Component	What	Lines
	<code>docs/openapi.json</code> , reports coverage	
<code>tests/test_openapi_contract.py</code>	Contract test — validates SDK types match OpenAPI schemas	~160

27.3 Coverage

Route File	Routes Wired	Skipped (returns raw dict / StreamingResponse)
<code>decision_routes.py</code>	4 (log, get, replay, scrub)	3 (stats, export, query)
<code>erasure_routes.py</code>	4 (issue, get, verify, fact)	2 (stats, list)
<code>governance_routes.py</code>	6 (stats, policies CRUD, logs)	4 (types, delete, evaluate, seed)
<code>orchestrator_routes.py</code>	2 (stats, verify-chain)	13 (agent/workflow CRUD — complex dicts)
<code>regulatory_routes.py</code>	4 (stats, generate, list, get)	2 (download endpoints)
<code>llm_routes.py</code>	3 (register, list providers, list tools)	4 (delete, tool register, seed)
<code>webhook_routes.py</code>	6 (CRUD + event types)	3 (delete, test, deliveries)
<code>sso_routes.py</code>	2 (providers, configure)	2 (authorize, callback)
Total	31 routes wired	33 remaining

[!NOTE] The remaining 33 routes return ad-hoc `dict` shapes that need individual response models. These will be wired incrementally as each route file is touched. The contract test tracks coverage percentage and enforces a minimum threshold.

27.4 Developer Experience

GET /docs → Swagger UI (interactive)

GET /redoc → ReDoc (publication-quality docs)

GET /openapi.json → Raw OpenAPI 3.1 spec (machine-readable)

27.5 Conformance Expansion

Sprint 12 expanded the conformance suite from **39** → **49 tests** across **17** → **21 phases**:

Phase	Tests	Coverage
Phase 18: SSE Streaming	2	<code>stream_workflow</code> , <code>stream_nonexistent_404</code>
Phase 19: Webhook CRUD	5	<code>register</code> , <code>list</code> , <code>event_types</code> , <code>cross_tenant</code> , <code>delete</code>
Phase 20: PDF Export	2	<code>pdf_download</code> , <code>pdf_missing_404</code>
Phase 21: SSO Providers	1	<code>sso_provider_list</code>

28. Advanced Monitoring — Sprint 13

Sprint 13 adds full observability infrastructure — the final prerequisite for production Kubernetes deployments.

28.1 Health Endpoints

Endpoint	Auth	Purpose
<code>GET /healthz</code>	None	Liveness probe — returns 200 if the process is running. Response includes <code>status</code> , <code>uptime_seconds</code> , <code>started_at</code> , <code>version</code> .
<code>GET /readyz</code>	None	Readiness probe — checks downstream dependencies (DB connectivity, pool status, store manager, all cloud services). Returns 503 if degraded.

Endpoint	Auth	Purpose
GET <code>/v1/monitoring/stats</code>	API Key	Full system stats — pool gauges, store count, Prometheus metric summary. Powers the portal Monitoring tab.

All health/metrics endpoints bypass auth middleware (`_SKIP_AUTH_PATHS`) so Kubernetes, Railway, and load balancer probes can reach them without an API key.

28.2 Prometheus Metrics

The `/metrics` endpoint serves the standard Prometheus exposition format using `prometheus-client`. **18 metric definitions** across three categories:

HTTP RED metrics (auto-instrumented via `PrometheusMiddleware`):

Metric	Type	Labels
<code>grafomem_http_requests_total</code>	Counter	method, path_template, status
<code>grafomem_http_request_duration_seconds</code>	Histogram	method, path_template, status
<code>grafomem_http_requests_in_progress</code>	Gauge	method
<code>grafomem_http_response_size_bytes</code>	Histogram	method, path_template

Business metrics (manually instrumented at event points):

Metric	Type	Labels
<code>grafomem_governance_evaluations_total</code>	Counter	result (allow/deny/escalate)
<code>grafomem_workflows_total</code>	Counter	status (completed/failed/terminated)

Metric	Type	Labels
<code>grafomem_workflow_duration_seconds</code>	Histogram	mode
<code>grafomem_tokens_consumed_total</code>	Counter	model_id
<code>grafomem_decisions_logged_total</code>	Counter	model_id
<code>grafomem_memory_operations_total</code>	Counter	operation (write/retrieve/delete/superse de/batch_write)
<code>grafomem_erasures_certificates_total</code>	Counter	—
<code>grafomem_webhooks_dispatched_total</code>	Counter	event_type, success
<code>grafomem_sso_logins_total</code>	Counter	provider

Infrastructure gauges (refreshed on each `/metrics` scrape):

Metric	Type
<code>grafomem_db_pool_size</code>	Gauge
<code>grafomem_db_pool_available</code>	Gauge
<code>grafomem_db_pool_waiting</code>	Gauge
<code>grafomem_stores_active</code>	Gauge
<code>grafomem_uptime_seconds</code>	Gauge

28.3 Path Normalization

To prevent Prometheus label cardinality explosion, all URL paths are normalized before recording. UUIDs, hex IDs, and numeric segments are collapsed to `{id}`. Static file paths (`/portal`, `/landing`) are excluded entirely.

28.4 Graceful Degradation

If `prometheus-client` is not installed, all metrics become silent no-ops (`_NoOpMetric`). This ensures the metrics module is optional and doesn't break non-cloud deployments or testing environments.

28.5 Portal Monitoring Tab

New portal tab: **Monitoring** — a real-time dashboard with 5-second auto-refresh that displays:

- **System Status:** health status, uptime, version, active stores
- **Connection Pool:** size, available, waiting, utilization bar with color-coded thresholds
- **Governance Activity:** allow/deny/escalate counters
- **Workflow Activity:** completed/failed/terminated counters
- **Operations Since Boot:** memory ops, decisions, erasure certs, webhooks, SSO logins
- **Health Endpoints Reference:** table of all monitoring endpoints with auth requirements

Polling starts when the tab is activated and stops when navigating away (to avoid unnecessary API calls).

28.6 Conformance Expansion

Phase	Tests	Coverage
Phase 22: Health Checks	2	<code>health_liveness</code> (200 + status=ok), <code>health_readiness</code> (200 + dependency checks)

Suite total: **51/51** across **22 phases**.

28.7 Sixth Production Gate

With Sprint 13, a sixth production gate is closed:

Gate	What	Status
Gate 1	Conformance 51/51 (Mock + Live)	✓
Gate 2	GMP self-conformance M8 = 1.000	✓

Gate	What	Status
Gate 3	HITL resume lifecycle (two-sided)	✓
Gate 4	OpenAPI contract test (SDK ↔ spec)	✓
Gate 5	SDK integration 32/33	✓
Gate 6	Prometheus observability + health probes	✓

End of document. Version 2.5 — 20 sprints complete. All six production gates closed: 51/51 conformance (mock) + 50/50 v3 governed-services + 12/12 gcrumbs = 113 gates, GMP self-conformance M8=1.000, HITL resume lifecycle validated two-sided, OpenAPI contract test validates SDK ↔ spec alignment, Prometheus observability with 18 metrics + health probes. Python SDK shipped with 32/33 integration tests passing against live sandbox. 112/112 unit tests green. Sprints 17–20 deliver multi-provider LLM support — all three providers have live conformance evidence (OpenAI 39-test baseline; Anthropic 49/51; Gemini 48/51) — CrewAI/AutoGen SDK adapters, Continuous Assurance engine (drift detection + baselines), and RoutingPool horizontal scaling with read-replica routing (June 1, 2026).

29. Sprint 14 — E2E Pipeline + GNS Dogfood Flight

Sprint 14 closes the whitepaper §9 commitment: **"the dogfood flight has been flown."**

Phase A — Five-Stage E2E Pipeline Test

A single test (`tests/test_v3_e2e_pipeline.py`) exercises R2→R1→R3→R4→R5 as one governed flow:

Stage	Capability	Assertion
1	R2 Seal corpus	Clearance = <code>cleared</code> , Merkle root present, inclusion proof verified

Stage	Capability	Assertion
2	R1 Register artifact	Content-addressed ID, Ed25519 receipt verified
3	R3 Issue Landing Certificate	Certificate verified (non-vacuous), R3→R1 auto-certify cross-link confirmed
4	R4 Compose	Receipt verified, <code>composed_artifact()</code> returns registrable descriptor
5	R5 World-model	Object + action types registered, governed action receipt signed and verified

All **6 stages passed** (cross-link assertion is stage 3'). Each step feeds state to the next — R2's `provenance_block()` feeds R3, R3 auto-certifies back to R1, R4 consumes the certified artifact.

Phase B — GNS Dogfood Ingestion

The actual GRAFOMEM codebase (`tests/test_dogfood_ingestion.py`) ingested through the full pipeline:

Step	What happened
R2	95 Python files, 31,893 LOC scanned → BLAKE2b content-hashed → Merkle tree sealed → Article-10 attestations (representativeness, bias examination, data gaps)
R1	<code>oci://gns-foundation/grafomem-kb:v3.0</code> registered with 10 representative layers, receipt verified
R3	Landing Certificate issued under <code>release</code> tier / <code>camilo@ulissy.app</code> , conformance 101/101, 10 permitted actions → verified (non-vacuous) , R3→R1 auto-certify 

Step	What happened
R4	KB + <code>gemini-2.5-pro@2026-05</code> composed as <code>grafomem-assistant:v3.0</code> , receipt verified
R5	GNS ontology: 6 object types (Protocol, Capability, ConformanceReport, Adapter, Agent, DelegationCertificate), 5 link types (implements, tested_by, adapts, runs_on, authorized_by), 5 governed action types (deploy_agent, run_conformance, issue_delegation, register_adapter, revoke_delegation)
Action	<code>deploy_agent</code> invoked at <code>release</code> tier → signed receipt 
Negative	<code>issue_delegation</code> at <code>basic</code> tier → correctly rejected (<code>basic < root</code>) 

Attestation

An independent party can verify, from the Landing Certificate and public keys alone:

- **What** was deployed: `oci://gns-foundation/grafomem-kb:v3.0` (content-addressed)
- **From where**: Merkle-rooted corpus of 95 files with provenance attestations
- **Under whose authority**: `camilo@ulissy.app` at `release` tier via `gns-root-2026`
- **Which actions are permitted**: 10 governed actions, each tier-gated

The mechanism the whitepaper §9 described as pending is now proven by running code.

30. Sprint 15 — gcrumbs Merkle-epoch Anchor

Sprint 15 implements the production gcrumbs service — a breadcrumb chain + Merkle epoch anchor that covers **all governance events**, not just orchestrator steps.

30.1 Architecture

`GcrumbsService` in `cloud/gcrumbs.py` (~420 lines):

- **Breadcrumb chain:** BLAKE2b hash chain with `prev_id` linking. Each breadcrumb stores `event_type`, `payload`, `source_type`, and a `payload_canon` BYTEA column (canonical JSON bytes stored at append time, never re-serialized from JSONB).
- **Merkle epochs:** Sealed via `roll_epoch()`. Each epoch computes a cumulative Merkle root over all breadcrumbs up to that point. Ed25519-signed.
- **Inclusion proofs:** Any breadcrumb can prove membership in any epoch that contains it.
- **Atomic sequencing:** `pg_advisory_xact_lock` ensures breadcrumb `seq` is monotonically increasing even under concurrent appends.

30.2 Separate Merkle Trees

[!WARNING] The gcrumbs Merkle tree is **NOT** R2's domain-separated tree:

- **gcrumbs:** `node = b2_256((left_hex + right_hex).encode())` — hex-string concatenation
- **R2 provenance:** `node = blake2b(\x01 + left_bytes + right_bytes)` — domain-separated bytes

A shared `merkle.py` would produce wrong roots. B0 proves this is correct by reproducing the CDP artifact byte-for-byte.

30.3 Service Wiring

Breadcrumbs are emitted internally by all 5 governed services + erasure. `POST /breadcrumbs` is deliberately absent — append is not caller-reachable.

Service	Event type	Emitted at
R3 Landing	<code>landing_certificate</code>	<code>_anchor()</code> after certificate issuance
R5 World-model	<code>action:<name>:ok</code>	<code>_emit_receipt()</code> after action invocation
R4 Composition	<code>composition</code>	<code>_emit()</code> after governed compose
R2 Customs	<code>customs:seal</code>	<code>register_corpus()</code> after seal
Erasure	<code>erasure:issued</code>	<code>issue_certificate()</code> after erasure

All hooks are best-effort (`try/except` → `log warning`), so a broken hook never blocks the parent operation.

30.4 Conformance

Gate	What	DB	Result
B0 (9 checks)	CDP artifact reproduction — crypto only, imports production <code>_leaf/_merkle/b2_128/canon/verify_inclusion</code>	No	9/9 
B1	append + roll → epoch sealed	Yes	
B2	Merkle root recomputation from DB breadcrumbs	Yes	
B3	epoch Ed25519 signature verified	Yes	
B4	cumulative prefix (epoch 2 includes epoch 1)	Yes	
B5a	tamper payload → chain reports <code>tampered</code>	Yes	
B5b	tamper epoch signature → verification fails	Yes	
B6	inclusion proof non-vacuous + verified	Yes	
B7	cross-service breadcrumbs (all 5 families) in one epoch	Yes	

Gate	What	DB	Result
B8	empty epoch refused (GcrumbsError)	Yes	✓
B9	chain verification detects tampered prev_id	Yes	✓
B10	genesis prev_id = '0' * 32	Yes	✓

Total: 9 + 11 = **20 checks** (12 gates, B5 split into 2). Existing 50 R1–R5 gates confirmed no regression.

30.5 Database Schema

```
CREATE TABLE gcrumbs_breadcrumbs (

    breadcrumb_id TEXT PRIMARY KEY, -- BLAKE2b-128(prev_id // leaf_hash)

    tenant_id TEXT NOT NULL,

    seq INTEGER NOT NULL, -- monotonic per tenant

    event_type TEXT NOT NULL, -- landing_certificate, action:*, etc.

    payload JSONB NOT NULL, -- event payload

    payload_canon BYTEA NOT NULL, -- canonical bytes (float safety)

    prev_id TEXT NOT NULL, -- chain link ('0' * 32 for genesis)

    source_type TEXT,

    signature TEXT, -- Ed25519 hex

    sealer_pubkey TEXT, -- Ed25519 public key hex

    created_at TIMESTAMPTZ DEFAULT NOW()

);
```

```
CREATE TABLE gcrumbs_epochs (

    epoch_id    TEXT PRIMARY KEY,  -- BLAKE2b-128(root // epoch_number // n_leaves)

    tenant_id   TEXT NOT NULL,

    epoch_number INTEGER NOT NULL,

    merkle_root TEXT NOT NULL,     -- BLAKE2b-256 hex

    n_leaves    INTEGER NOT NULL,  -- cumulative

    signature   TEXT NOT NULL,     -- Ed25519 over epoch_id

    sealer_pubkey TEXT NOT NULL,

    sealed_at   TIMESTAMPTZ DEFAULT NOW()

);
```

31. Sprint 16 — Enterprise Infrastructure

[!NOTE] Sprint 16 adds SAML 2.0 federated authentication, upgrades the Portal SSO UI with social login buttons, and completes the database pool migration across all 24 cloud services.

31.1 SAML 2.0 Service Provider

File: `sso_provider.py` (extended) **Table:** `saml_configs` **Endpoints:** 4 under `/v1/portal/sso/saml/`

Feature	Implementation
IdP metadata	XML auto-discovery via URL or raw paste
SP metadata	Generated at <code>/v1/portal/sso/saml/metadata</code>
SP-initiated SSO	<code>AuthnRequest</code> with DEFLATE + base64 encoding, HTTP-Redirect binding

Feature	Implementation
Assertion Consumer Service	<code>/v1/portal/sso/saml/acs</code> — HTTP-POST binding, NameID + attribute extraction
Configuration	<code>SAMLConfig</code> dataclass, <code>saml_configs</code> table
Attribute mapping	Configurable SAML attribute OIDs → email/name fields

31.2 Portal SSO UI

Feature	Implementation
Social login buttons	Google, Microsoft, GitHub with SVG brand icons
Provider auto-discovery	<code>/v1/portal/sso/providers</code> → show/hide buttons dynamically
Browser redirect flow	Callback → <code>/portal?token=...&email=...</code>
Styling	Glassmorphism with provider-specific hover colors

31.3 Database Pool Migration

All 24 cloud services now accept a `pool=None` parameter, completing the migration started in Sprint 11:

Pattern	Services	Mechanism
Context-manager services	6	<code>_get_conn()</code> + <code>_put_conn()</code> pair
Cached-connection services	18	<code>_get_conn()</code> prefers pool, <code>close()</code> guards pool

app.py wiring: `pool = getattr(app.state, 'db_pool', None)` passed to all constructors.

Backward compatible: `pool=None` falls back to lazy `psycpgp.connect()` — no pool required for development or testing.

32. Sprint 17 — Anthropic / Gemini Provider Support

[!NOTE] Sprint 17 adds Anthropic (Claude Opus 4) and Gemini (2.5 Pro) as provider options alongside existing OpenAI and Mock. **What shipped:** adapter plumbing, CLI flags (`--anthropic` / `--gemini`), 16 DB-free unit tests, and live conformance runs for all three providers. OpenAI gpt-4o-mini runs at the 39-test baseline; Anthropic Claude Opus 4 at **49/51**; Google Gemini 2.5 Pro at **48/51**. No live run is a clean 51 — non-deterministic LLM output produces expected divergences in replay and tool-calling phases.

[!WARNING] **Live conformance evidence is provider-specific.** The 113-gate figure (51 platform + 50 v3 + 12 gcrumbs) is the mock/local total. Per-provider live evidence must always cite the specific score and run artifact.

32.1 Optional Dependencies

File: `pyproject.toml` (updated)

5 new extras added:

```
anthropic = ["anthropic>=0.30"]
```

```
gemini = ["google-genai>=1.0"]
```

```
llm = ["openai>=1.0", "anthropic>=0.30", "google-genai>=1.0"]
```

```
crewai = ["crewai>=0.60,<2.0"]
```

```
autogen = ["autogen-agentchat>=0.4"]
```

32.2 E2E Provider Support

File: `tests/sandbox_e2e_v2.py` (modified)

Flag	Provider	Model	Env Var
(default)	Mock	<code>mock-model</code>	—

Flag	Provider	Model	Env Var
<code>--live</code>	OpenAI	<code>gpt-4o-mini</code>	<code>OPENAI_API_KEY</code>
<code>--anthropic</code>	Anthropic	<code>claude-opus-4-20250514</code>	<code>ANTHROPIC_API_KEY</code>
<code>--gemini</code>	Gemini	<code>gemini-2.5-pro</code>	<code>GOOGLE_API_KEY</code>

32.3 Provider Tests

File: `tests/test_llm_providers.py` (16 tests, all DB-free)

Verifies enum completeness, request/response data models, mock adapter determinism, role detection, tool calling, token counting, input hash embedding, and import error messages for all 3 providers.

[!IMPORTANT] The 16 unit tests verify adapter plumbing (no network calls). Live conformance evidence:

- **OpenAI** (`--live`): 39/39 baseline, 1104 tokens, replay diverged confidence=0.54
- **Anthropic** (`--anthropic`): **49/51**, Claude Opus 4 (`claude-opus-4-20250514`)
- **Gemini** (`--gemini`): **48/51**, Gemini 2.5 Pro

No provider achieves a clean 51/51 live — this is expected. LLM non-determinism causes replay divergence (confidence < 1.0) and occasional tool-calling format mismatches.

33. Sprint 18 — CrewAI / AutoGen SDK Adapters

[!NOTE] Sprint 18 delivers governed memory and governance callback adapters for CrewAI and AutoGen, following the LangChain adapter pattern.

33.1 CrewAI Adapter (`sdk/src/grafoem/crewai/`)

File	Class	Purpose
<code>storage.py</code>	<code>GrafoemCrewStorage(Storage)</code>	<code>save()</code> / <code>search()</code> / <code>reset()</code> delegating to GRAFOMEM memory API
<code>callbacks.py</code>	<code>GrafoemGovernanceCallback</code>	<code>on_task_start()</code> pre-task governance eval, <code>on_task_end()</code> decision logging
<code>callbacks.py</code>	<code>GovernanceDeniedError</code>	Raised when governance denies execution (configurable via <code>deny_raises</code>)

33.2 AutoGen Adapter (`sdk/src/grafoem/autogen/`)

File	Class	Purpose
<code>memory.py</code>	<code>GrafoemAutoGenMemory</code>	<code>add_message()</code> / <code>get_context()</code> / <code>get_messages()</code> / <code>clear()</code>
<code>hooks.py</code>	<code>GrafoemGovernanceHook</code>	<code>pre_send()</code> returns None on deny (blocks message), <code>post_receive()</code> logs decisions

33.3 Tests

- `test_crewai_adapter.py`: 11 tests (mock Storage base class via `sys.modules` patching)
 - `test_autogen_adapter.py`: 12 tests (no framework import needed)
-

34. Sprint 19 — Continuous Assurance

[!NOTE] Sprint 19 implements the Continuous Assurance engine — scheduled conformance checks, drift detection against baselines, and a full REST API.

34.1 AssuranceService (`cloud/assurance.py`, ~420 lines)

Tables: `assurance_schedules`, `assurance_runs`, `assurance_baselines` (3 new tables + indexes)

`run_check()` executes 5 checks:

1. **Health** — readiness probe via HealthChecker
2. **Database** — `SELECT 1` connectivity
3. **Metrics** — snapshot via `get_metrics_summary()`
4. **Chain integrity** — gcrumbs breadcrumb count for tenant
5. **Governance** — active policy count

Drift detection (`_detect_drift()`):

- **Regression:** check was passing in baseline, now failing
- **Metric anomaly:** 2x threshold on `error_rate` or `avg_latency_ms`

34.2 AssuranceScheduler (`cloud/scheduler.py`, ~120 lines)

- `asyncio.Task` per active schedule (run in executor for non-blocking DB calls)
- Webhook dispatch on drift/failure events
- 60-second backoff on errors
- Default interval: **every 60 minutes** (configurable 5–1440 min)

34.3 API Endpoints (11 under `/v1/assurance/`)

Schedule CRUD (4) + manual run trigger + run history (2) + baseline capture/get (2) + drift events + stats.

34.4 Tests

`test_assurance.py`: 14 DB-free tests covering data models, drift detection logic, row converters, and scheduler lifecycle.

35. Sprint 20 — Horizontal Scaling

[!NOTE] Sprint 20 adds read-replica routing to the database pool infrastructure, enabling horizontal read scaling without modifying any of the 25 cloud services.

35.1 RoutingPool (`cloud/db_pool.py`, appended)

```
pool = RoutingPool(primary_url)          # No replica configured

pool = RoutingPool(primary_url, read_url=url)  # Explicit replica

# Or: set GRAFOMEM_DB_READ_URL env var      # Env-var activated

pool.open()

conn = pool.getconn()                    # → primary

conn = pool.getconn(readonly=True)        # → replica (failover → primary)
```

Key properties:

- **Transparent:** `getconn()` backward-compatible — all 25 services work unchanged
- **Failover:** replica errors fall back to primary automatically with warning log
- **Env-var activated:** `GRAFOMEM_DB_READ_URL`, `GRAFOMEM_DB_READ_POOL_MIN`, `GRAFOMEM_DB_READ_POOL_MAX`
- **Stats:** `pool.stats` returns `{"primary": {...}, "replica": {...}}`

35.2 App Wiring

`app.py` changed from `DatabasePool(db_url)` to `RoutingPool(db_url)` — single-line swap. Logs `replica=True/False` at startup.

35.3 Tests

`test_read_replica.py`: 15 DB-free tests covering routing logic, failover, env-var activation, statistics, and lifecycle.

36. Phase 5 — Compliance & Enterprise Operations

[!NOTE] Phase 5 bridges the gap between academic cryptographic guarantees and production enterprise requirements, focusing heavily on GDPR/DORA compliance, Right-to-be-Forgotten (W9), observability, and automated disaster recovery.

36.1 The Right-to-be-Forgotten Pipeline (Sprint 26)

To satisfy W9 Erasure without blocking the hot path:

- **Decoupled Deletions:** `ON DELETE CASCADE` was dropped. Deletions flag records as `erasure_pending`.
- **ErasureSweeper Daemon:** Daemon deployed live to production as an independent Railway worker. The 60-minute GDPR erasure SLA is actively verified and sweeping orphaned embeddings successfully.
- **Protocol 3.4 Normative Contract:** Recomputable states (`declared`, `observed`) are cleanly separated from the trust layer.
- **Offline Verification:** Two-pass verification strictly enforces `failed` > `incomplete` > `enforced` precedence.

36.2 Enterprise Observability & SLAs

- **SLO Commitments:** Formally defined 99.9% uptime and p95 latency < 250ms boundaries. (Built/Defined, pending active alerting)
- **Prometheus Telemetry:** Hardened the observability stack with `grafomem_erasure_sweep_errors_total`. (Built/Defined, pending active alerting)
- **Runbooks:** Standard Operating Procedures (SOPs) written for on-call engineers.

36.3 Backup, DR, and Log Retention

- **W6 Restore Probe:** An automated script (`verify_restore_probe.py`) that audits restored databases. (Built but unverified against real snapshot)
- **SIEM Exporter:** A resilient background daemon (`siem_exporter.py`) that safely batches new `decision_records` and `gcrumbs_breadcrumbs` and streams them to an enterprise SIEM. (Code-complete and explicitly undeployed to prevent premature retention pruning).
- **180-Day Retention:** Exporter prunes archived logs older than 180 days after verification. (Built, pending production deployment)

Appendix: Live Status

[!NOTE] "LIVE" denotes a working, deployed code path validated locally and/or against a small live sample — not a battle-tested-at-scale claim. Conformance figures are mock/local unless a specific live run and provider are cited (see §32).

Component	Status	Evidence / Caveats
Local-first inference (BYOM)	LIVE	Local BYOM abstraction over REST; exercised with a llama-cpp-python backend.
Cloud managed inference	LIVE	Multi-provider (§32): OpenAI <code>gpt-4o-mini</code> (39-test baseline), Anthropic Claude Opus 4 (49/51), Gemini 2.5 Pro (48/51). No live run is a clean 51.
Action extraction	LIVE	Extracts <code>action_name</code> and <code>params</code> via strict schema forcing.
Declarative governance PEP	LIVE	PEP enforces DB-defined policies (e.g. <code>require_params</code> , <code>sandbox_financial_rules</code>); resilience mechanisms (failover, tool-deny, timeout, loop) validated two-sided in the sealed run (§17.1).
Tamper-evident receipts	LIVE	Ed25519 signatures generated and independently verifiable with bound production keys; tamper detection proven via negative tests.
Erasure / gcrumbs	LIVE	Erasure sweeper deployed as independent Railway worker; 60-minute GDPR deletion actively verified. SIEM Exporter code-complete. W6 restore probes built but unverified.